

# MACHINE LEARNING IN FINANCE

MODULE 5 | LESSON 2

---

## SUPPORT VECTOR MACHINES IN PRACTICE

---

**Reading Time** 20 minutes

**Prior Knowledge** Classification, Support Vector Machines, Decision Trees.

**Keywords** Hyperparameters, slack variable, returns, Receiver Operating Curve (ROC).

---

*In the previous lesson, we covered the methodology of Support Vector Machines. In this lesson, we apply it to a predictive analysis problem.*

### 1. Introduction

In this lesson, we will implement SVM on a trading strategy for the Luxembourg index (LUXXX) based on other country indices as well as technical indicators. The strategy is to take a long/short position when a 0.25% change in LUXXX return is predicted. The long/short position will be based on an upward or downward trending market. Keep in mind that this strategy does not take into account trading costs, but the lesson is designed to show the practicality of SVM in predictive analytics for financial problems.

We will begin by importing the necessary packages.

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn import svm
from sklearn.metrics import roc_auc_score, roc_curve
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.tree import DecisionTreeClassifier
```

The data containing the weekly closing price of indices can be found in the following csv file along with the dates. We load this data and thereafter view it using the head() command.

Should you store the csv file in the same location as the notebook, then use the uncommented cells below. Use the commented cells instead if you choose to store the csv in a different location. The "loc" variable is the path to the csv you need to specify.

```
In [ ]: # Loc = "ENTER YOUR FULL PATH TO LOCATION OF DATA FILE HERE"
# data_df = pd.read_csv(Loc+"/MScFE 650 MLF GWP Data.csv")
data_df = pd.read_csv("MScFE 650 MLF GWP Data.csv")
data_df.head(3)
```

The 'Date' column is in a string format; therefore, we convert it to a datetime format.

```
In [ ]: # Convert string to datetime
data_df["Date"] = pd.to_datetime(data_df["Date"])
```

Below is where we specify the target column. In our case, it is the LUXXX index. The returns of LUXXX need to be calculated since our strategy is based on the returns over the weeks. The remaining indices are converted to returns instead of their prices. This accomplishes two things, namely it scales the features into a common unit as well as captures movement in the prices instead.

```
In [ ]: # Set Target Index for predicting
target ETF = "LUXXX"

# Use returns instead of prices for other Indices
# Other Indices used as Index_features
ETF_features = data_df.loc[:, ~data_df.columns.isin(["Date", target ETF])]
data_df[ETF_features] = data_df[ETF_features].pct_change()

data_df[target ETF + "_returns"] = data_df[target ETF].pct_change()
```

The cell below calculates the Target column and converts it into a categorical binary variable. We denote 1 for the absolute returns of LUXXX exceeding 0.25% and 0 otherwise. In a downward or bear market, we look to short should the price decrease by more than this threshold and we take a long position in a bear market. The goal here is to predict a percent change more than a threshold. Note that we shift the target column by one week to align the predicted period with the period available for other predictors. The ML algorithm requires the label and feature/predictor values to be captured in the same row or observation.

```
In [ ]: # Create Target Column.
# Shift period for target column
data_df[target ETF + "_returns" + "_Shift"] = data_df[target ETF + "_returns"].shift(1)

# Strategy to take long position for anticipated returns of 0.5%
data_df["Target"] = np.where(
    (data_df[target ETF + "_returns_Shift"].abs() > 0.025), 1, 0
)
```

It is worth looking at any imbalance in the dataset. The proportion of 34.5% of the target class 1 does not indicate any imbalance.

```
In [ ]: # Checking target proportion
round(data_df["Target"].sum() / len(data_df), 4)
```

Now that we have our Target class, we can select which indices contribute to predicting the target class. We use the feature importance tool in `Sklearn` for a Decision Tree to

do this. We look at the cumulative importance for the features that make a contribution > 0.

```
In [ ]: # Preparing for Train/Test split

NoNaN_df = data_df.dropna()
X = NoNaN_df[ETF_features]

X = X.iloc[:, :] # .values
y = NoNaN_df.loc[:, "Target"] # .values

del NoNaN_df

# from sklearn.cross_validation import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_

# USE DECISION TREE FOR FEATURE IMPORTANCE
# Fitting Decision Tree Classification to the Training set

DTree = DecisionTreeClassifier(
    criterion="entropy",
    random_state=0,
    max_depth=8,
    min_samples_leaf=30,
    min_samples_split=20,
    # , class_weight=classweight
)
DTree.fit(X_train, y_train)

feature_importances = pd.DataFrame(
    DTree.feature_importances_,
    index=X.iloc[:, :].columns, # don't want the last Target column
    columns=["importance"],
).sort_values("importance", ascending=False)

feature_importances["Cumul_Imp"] = feature_importances.cumsum().iloc[:, :]
# If you want to see only those that explain a % of variation
pct_var = 1.0
feature_importances = feature_importances[(feature_importances["Cumul_Imp"] <= p
```

The cumulative importance shows only 4 indices that contribute, namely MSCI KOREA, DENMARK, FRANCE, and NORWAY. This reduces the features significantly and will also reduce the run time when doing hyperparameter tuning.

```
In [ ]: # PLOT IMPORTANCE

# Example data
cols = feature_importances.index
y_pos = np.arange(len(cols))
performance = feature_importances.importance

df2plot = pd.DataFrame(data=performance, index=feature_importances.index)
df2plot["Variable"] = feature_importances.index
df2plot = df2plot[df2plot["importance"] > 0.000]

print(df2plot)
```

```
f, ax = plt.subplots(figsize=(6, 5))

sns.barplot(
    x="importance", y="Variable", data=df2plot, label="Variable Ranking", color=
)

ax.set(xlim=(0, 1), ylabel="", xlabel="importance")

for p in ax.patches:
    width = p.get_width()
    ax.text(
        width + 0.05,
        p.get_y() + p.get_height() / 2.0 + 0.2,
        "{:1.2f}".format(width),
        ha="center",
    )
```

**Figure 1: Feature Importance Contribution of Four Predictors**

```
In [ ]: # Indices to add value in prediction
ETF_ImpFeatures = []
for i in df2plot["Variable"].values:
    ETF_ImpFeatures.append(i)

ETF_ImpFeatures
```

## 2. Create Technical Indicators

Among the first technical indicators are the moving average crossovers of short- (SMA\_5) and long-term (SMA\_15). We use the ratio of (SMA\_15/SMA\_5) to capture the relationship as one feature and any crossovers. We will rely on the SVM model to capture the relationship between this feature and the target class.

```
In [ ]: data_df["SMA_5"] = data_df[target ETF].rolling(5).mean()
data_df["SMA_15"] = data_df[target ETF].rolling(15).mean()
data_df["SMA_ratio"] = data_df["SMA_15"] / data_df["SMA_5"]

# Can drop SMA columns since not needed anymore.
data_df.drop(["SMA_5", "SMA_15"], axis=1, inplace=True)
```

The next technical indicator we look at is the relative strength index (RSI), which is a momentum oscillator. This indicator measures the magnitude of recent price changes. The target price is usually considered overbought when the RSI > 70% and oversold when < 30%. This is based on the price change of LUXXX before the week of prediction.

```
In [ ]: # shift the price of the target by 1 unit previous in time
data_df["Diff"] = data_df[target ETF] - data_df[target ETF].shift(1)
data_df["Up"] = data_df["Diff"]
data_df.loc[(data_df["Up"] < 0), "Up"] = 0

data_df["Down"] = data_df["Diff"]
data_df.loc[(data_df["Down"] > 0), "Down"] = 0
data_df["Down"] = abs(data_df["Down"])

data_df["avg_5up"] = data_df["Up"].rolling(5).mean()
```

```

data_df["avg_5down"] = data_df["Down"].rolling(5).mean()

data_df["avg_15up"] = data_df["Up"].rolling(15).mean()
data_df["avg_15down"] = data_df["Down"].rolling(15).mean()

data_df["RS_5"] = data_df["avg_5up"] / data_df["avg_5down"]
data_df["RS_15"] = data_df["avg_15up"] / data_df["avg_15down"]

data_df["RSI_5"] = 100 - (100 / (1 + data_df["RS_5"]))
data_df["RSI_15"] = 100 - (100 / (1 + data_df["RS_15"]))

data_df["RSI_ratio"] = data_df["RSI_5"] / data_df["RSI_15"]

# Can drop RS Calc columns columns
data_df.drop(
    ["Diff", "Up", "Down", "avg_5up", "avg_5down", "avg_15up", "avg_15down"],
    axis=1,
    inplace=True,
)

```

Rate of change is the percentage change in price over a 15-week period. It is a long-term rate of change and is also a momentum indicator. It is also used to spot overbought and oversold instances.

```
In [ ]: data_df["RC"] = data_df[target ETF].pct_change( periods=15)
```

We now combine all features, that is the indices' price change as well as the technical indicators.

```
In [ ]: # all_feats
ETF_ImpFeatures.append("SMA_ratio")
ETF_ImpFeatures.append("RSI_ratio")
ETF_ImpFeatures.append("RC")
```

We are now in a position to train our model. To begin, we perform the train/test split of the data into an 80/20 split.

```
In [ ]: # Train/Test split
# Train/Test split. No NaNs in the data.
NoNaN_df = data_df.dropna()
X = NoNaN_df[ETF_ImpFeatures]

X = X.iloc[:, :] # .values
y = NoNaN_df.loc[:, "Target"] # .values

del NoNaN_df

# from sklearn.cross_validation import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

To determine the best kernel and slack variable value, we do a grid search method using `GridSearchCV`. This will explore the specified hyperparameter space for the optimal values over five cross validations.

```
In [ ]: # defining parameter range
param_grid = {
```

```

    "C": [0.01, 0.1, 1, 10, 100, 1000],
    #             'gamma': [1, 0.1, 0.01, 0.001, 0.0001],
    "kernel": ["rbf", "linear", "poly"],
}

grid = GridSearchCV(svm.SVC(), param_grid, refit=True, verbose=3, cv=5)

# fitting the model for grid search
grid.fit(X_train, y_train)

```

The best hyperparameters are specified below. We can see that the best kernel to use for this dataset is the polynomial kernel with a slack variable value of 10.

```

In [ ]: # print best parameter after tuning
print(grid.best_params_)

# print how our model looks after hyper-parameter tuning
print(grid.best_estimator_)

```

Finally, we can train our SVM model for the specified hyperparameters and compare it to an untuned decision tree classifier.

```

In [ ]: # Train with Tuned SVM
# Create a svm Classifier
clf_tuned = svm.SVC(
    random_state=0,
    C=grid.best_params_["C"],
    kernel=grid.best_params_["kernel"],
    probability=True,
)

clf_tuned.fit(X_train, y_train)

# Fitting Decision Tree Classification to the Training set as well

clf_tree = DecisionTreeClassifier(
    criterion="entropy",
    random_state=0,
    max_depth=8,
    min_samples_leaf=30, # DM
    min_samples_split=20,
    #                                     , class_weight=classweight
) # DM
clf_tree.fit(X_train, y_train)

```

From the results below, we see that the SVM model is significantly better than a random guess; however, the tree-based approach performed slightly better with a 4% increase in AUC.

### 3. Performance

```

In [ ]: # Performance

# predicted probabilities generated by sklearn classifier

```

```

y_pred_proba = clf_tuned.predict_proba(X_test)
y_pred_probatree = clf_tree.predict_proba(X_test)

# SVM ROC dependencies
fpr, tpr, _ = roc_curve(y_test, y_pred_proba[:, 1])
# TREE ROC dependencies
fpr_tree, tpr_tree, _ = roc_curve(y_test, y_pred_probatree[:, 1])

auc = round(roc_auc_score(y_test, y_pred_proba[:, 1]), 4)
auc_tree = round(roc_auc_score(y_test, y_pred_probatree[:, 1]), 4)

# SVM Model
plt.plot(fpr, tpr, label="SVM, auc=" + str(auc))
# Tree model
plt.plot(fpr_tree, tpr_tree, label="Tree, auc=" + str(auc_tree))
# Random guess model
plt.plot(fpr, fpr, "-", label="Random")
plt.title("ROC")
plt.ylabel("TPR")
plt.xlabel("FPR")

plt.legend(loc=4)
plt.show()

```

**Figure 2. ROC Curve for Three Models**

## 4. Conclusion

This lesson focused on implementing the SVM model in Python on a predictive analytical problem, specifically a trading strategy for long positions of an index. We also included technical indicators as part of the predictors. We found that the SVM provides better results than a random guess, and the results are not too far off from those of a decision tree classifier. As an exercise, you can adjust this notebook to look at a hyperparameter-tuned decision tree classifier as well. In the next lesson, we will look at a model that is very effective on non-linear data, namely an artificial neural network.

# MACHINE LEARNING IN FINANCE

MODULE 5 | LESSON 3

---

## NEURAL NETWORKS

---

|                     |            |
|---------------------|------------|
| <b>Reading Time</b> | 20 minutes |
|---------------------|------------|

|                        |                                      |
|------------------------|--------------------------------------|
| <b>Prior Knowledge</b> | Linear and Non-linear data, gradient |
|------------------------|--------------------------------------|

|                 |                                                    |
|-----------------|----------------------------------------------------|
| <b>Keywords</b> | Activation function, Loss function, Neuron, Layers |
|-----------------|----------------------------------------------------|

---

*In the previous lesson, we covered support vector machines, which can handle both linear and non-linear data provided the appropriate kernel is used. In this lesson, we will cover a neural network predictor that is effective for both linear and non-linear data.*

### 1. Neural Network Basics

Neural networks (NNs) are often described as a way to replicate the human brain with a computer. When we think about the process of identifying an image, such as an image of a dog, we first view the image with our eyes thereby sending inputs to the brain. The next step would be the brain analyzing the image based on what we've seen about dogs in the past. Finally, the brain has made a decision/conclusion that this is an image of a dog. To summarize this process, we could have:

Input -> analyze -> decision.

This task is quite simple for us; however, to get a computer to perform the same image recognition task for a dog is not trivial. The process is similar in that the computer undertakes a similar process:

Input -> process/calculations -> output.

Let's look at an example to understand the process of a neural network to discover the relationship between inputs and outputs. We'll consider the simple problem of converting mass from pounds (lbs) to kilograms (kg). Suppose we didn't know the formula for the conversion, and all we had to go on was the observation:

---

| kg | lbs    |
|----|--------|
| 50 | 110.23 |

---

We make a critical assumption here: that the relationship between kg and lbs is linear. I know it's a good guess since we know the formula is  $\text{lbs} = 2.2046 \times \text{kg}$ , but often, we don't know the relationship. So we've made our guess that the relationship is  $\text{lbs} = c \times \text{kg}$ , where  $c$  is the slope. This linear equation is what is called the *activation function* in neural networks. We begin by making a guess about  $c=1.5$ , and with this guess, our lbs estimate is  $\text{lbs} = 1.5(50) = 75$  lbs. we therefore seem to have underestimated by  $110.23 - 75 = 35.23$  lbs. Okay, so we guess again, going higher for  $c$  based on the feedback from our first guess, and go with  $c=3$ , resulting in our second estimate of  $50(3) = 150$  lbs. It seems we overestimated this time by  $39.77$  lbs. We therefore need to go with a value of  $1.5 < c < 3$ . We do this iteratively until we get to a value of  $c=2.2$ , which results in  $110$  lbs. Pretty close. We can decide this is close enough, therefore settling on  $c=2.2$ . We can summarize the iterations below,

Table 1: Summary of Iterations to Find Optimal Slope  $c$ .

| iteration | $c$ | lbs estimate | (Actual-predicted) |
|-----------|-----|--------------|--------------------|
| 1         | 1.5 | 75           | 35.23              |
| 2         | 3.0 | 150          | -39.77             |
| 3         | 2.5 | 125          | -14.77             |
| 4         | 2.2 | 110          | 0.23               |

The last column provided insight into how far away our guess was from the actual. This is an example of a *loss function* in a NN. The magnitude of the difference between the actual and estimate signals how we need to adjust our next guess for  $c$ . Think of the difference being large resulting in our next guess of  $c$  being largely different from the previous one. This type of feedback is what we call the *optimizer* in a NN. With this example, we more or less get the workings or flow of a NN. This can be summarized in figure 1 below. You'll notice a 'weight' component in the diagram, which we will get to in the next section. To do this, we will look at the architecture of the NN.

Figure 1: Flow Diagram of Neural Network Operation

## 2. Network Architecture

A neural network's architecture is shown in figure 2. The hidden layer is where the processing takes place, and do keep in mind that we can have more than one hidden layer, but for now, we keep it simple.

Figure 2: Architecture of Basic Neural Network

If you look at the connections (edges) between the input layer and hidden layer, you will notice all red circles (input nodes) are connected to all the blue circles (Hidden layer nodes). We call this type of connection a *dense* layer. Think of each input node as a feature feeding information into the NN, for example in a credit risk problem, one input node could be Age with another being Salary. Hence, the number of nodes in the input layer is the number of features or predictors. The number of nodes in the output layer will depend on the target variable. If the target were a binary class variable, then one node would be sufficient in the output layer since it returns the probability of being in the target class, e.g., probability of defaulting on a loan. For a regression-type problem, like the lbs to kg conversion discussed earlier, the output node would return the continuous value of the target variable, i.e., lbs.

### 3. Forward and Backward Propagation

NNs achieve predictions via a process called propagation. At each forward step, there are matrix multiplications taking place. Computers are relatively quick at doing these calculations, and it is worth understanding the process behind it. To begin, we must first identify the dimension of inputs at the start, i.e., at the input layer. We mentioned earlier that the number of nodes at the input layer corresponds to the number of predictors or features. This is the number of columns of the input matrix, and the number of rows corresponds to the number of observations,  $N$  say. We thus have a  $N \times 3$  matrix of inputs for figure 2. In general, let's denote the number of features as  $p$  so figure 2 would have  $p=3$ . We shall denote the layers as follows:

1. Input layer:  $X_0$
2. Hidden layer:  $X_1$
3. Output layer:  $Y$

We'll touch on some linear algebra here but nothing too intense other than matrix multiplication. In the forward propagation step moving from  $X_0$  to  $X_1$ , we have the following calculations taking place, which can be represented as matrix multiplication,

$$X_1 = f(X_0 w_0 + b_0), \tag{1}$$
 where  $w_0$  is a matrix of  $p \times 4$  *weights* since the hidden layer has 4 nodes, and  $f$  is a user specified activation function. The term  $b$  is a bias term that enables the activation function to shift, which allows more flexibility of the algorithm to fit the data. The activation function, in addition to specifying the linearity/non-linearity of data, also decides on which node to activate by controlling the value of the weights. For instance, suppose Age was not a strong predictor of defaulting on a loan, then the weight for that node would be small. To sequentially move to the output layer, we then have the following calculation,

$$Y = f(X_1 w_1 + b_1) = f(f(X_0 w_0 + b_0) w_1 + b_1), \tag{2}$$

by substituting (1) for  $X_1$ . The dimension of  $w_1$  will be  $4 \times 2$  since we have 4 nodes in the hidden layer and 2 nodes in the output layer. This sequential process of matrix multiplication results in the following dimension changes:

- $X_0 \times w_0$ :  $(N \times p) \times (p \times 4) = N \times 4$  (Think of  $p$  canceling each other out)
- $X_1 \times w_1$ :  $(N \times 4) \times (4 \times 2) = N \times 2$

The dimensions of the weights and layers is necessary when specifying the architecture of the NN in a programming language, such as Python. Let's look at two videos that will help illustrate the inputs and outputs at the hidden nodes. Inputs of the hidden nodes can be shown in the video below.

```
In [ ]: from IPython.display import VimeoVideo  
  
VimeoVideo("782624672", h="60b0362998", width=600)
```

[Access Video Transcript here](#)

Outputs of hidden nodes can be show in this video:

```
In [ ]: VimeoVideo("782999865", h="2294f6c7e5", width=600)
```

[Access Video Transcript here](#)

Thus far, we have shown the forward propagation process to achieve outputs; however, it is the backward propagation process that enables the NN to learn and improve. This is the feedback process from the loss function to the optimizer, which eventually tells the algorithm how the weights should be adjusted. Ultimately, it is the weights, and some bias term, that will be adjusted to achieve optimal performance. A great benefit of NNs is that we do not have to restrict ourselves to just one hidden layer, and by increasing the layers, we increase our chances of solving non-linear data relationships. There is a problem that NNs face with this increase in layers, which is called the *vanishing gradient problem*.

Refer to the video below on how the bias is optimized using derivatives.

```
In [ ]: VimeoVideo("785293805", h="a725197abc", width=600)
```

[Access Video Transcript here](#)

## 4. Vanishing Gradient

We've briefly mentioned the role of the activation function in section 1. There are various activation functions to choose from with each having their benefits. The reader is directed to (Nwankpa 7) for a brief list of these functions. The sigmoid activation function is a popular function used in binary classification problems. It squashes the output into a small range between 0 and 1. The functional form may look familiar to you if you've worked with a logistic regression model as it is calculated as follows,

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

What this tells us is that for a large change in  $x$  or input, we have a small change in output. But we know that the gradient is given by,

$$\text{Gradient} = \frac{\text{change in output}}{\text{change in input}}$$

hence this implies the gradient tends to a small value for the sigmoid function. The connection of the sigmoid function to the loss function,  $L$  say, is that we know our output depends on the activation function. The more layers we have in our network, the more activation functions are required in the process. The idea of the backpropagation from table 1 was to minimize  $L$  or the difference in the last column through feedback. Calculus is very handy when we want to minimize some function  $f$  with respect to its input  $x$ . The process entails determining the value of  $x$ , say  $x'$ , that makes the derivative or gradient of  $f = 0$ . Backpropagation thus entails finding the derivative of the initial layer by multiplying the derivatives of each layer moving from the final to the initial layer, a process known as the chain rule. Thus, the problem arises where this product tends to zero with many layers, especially with the sigmoid activation function. This prevents optimal updating of the weights and bias.

Improvements to NNs have been made since 2009 to overcome this problem such as better activation functions like ReLu, better weight initialization schemes, and improved optimization schemes. NNs have come a long way since their inception, and with GPUs available, the speed of training these algorithms has been greatly improved.

## 5. Conclusion

In this lesson, we briefly explained the process of a NN. We mentioned its components such as inputs, hidden and output layers consisting of nodes, the forward and backward propagation process, and the role of the activation, loss, and optimizer in the operation of the algorithm. Neural networks also face problems such as vanishing gradients and overfitting if not tuned correctly. We'll look at an example of this in the next lesson where we train a neural network on a financial problem.

### References

Nwankpa, Chigozie. "Activation Functions: Comparison of Trends in Practice and Research for Deep Learning." *arXiv Preprint arXiv*, 1811.03378, 2018, pp. 5–8, [arxiv.org/pdf/1811.03378.pdf](https://arxiv.org/pdf/1811.03378.pdf).

# MACHINE LEARNING IN FINANCE

MODULE 5 | LESSON 4

---

## NEURAL NETWORKS IN FINANCE

---

**Reading Time** 25 minutes

**Prior Knowledge** Neural Network architecture, Economic data

**Keywords** Keras, TensorFlow, Layers, Mean Square Error, Mean Absolute Error

---

*The previous lesson covered the workings and architecture of a neural network. In this lesson, we put this into practice by exposing the reader to additional data preparation and model compilation.*

### 1. Predictive Problem

In this lesson, we will look at predicting monthly returns on the Dow Jones Industrial (DJI) Futures, specifically the Mini DJI Futures with ticker YM=F. The predictors we use are economic indicators from the FRED economic database. You may be familiar with this site from the Financial Data course. We consider five economic indicators, namely:

1. Average hourly earnings of all employees
2. All Employees Manufacturing
3. Producer Price Index by Industry
4. Durable goods New Orders
5. Manufacturers' New Orders: Total Manufacturing

Economic data can be found on this website <https://fred.stlouisfed.org/> and made available using an API key, which you can obtain by signing up with an account. Let's begin by importing the Economic data using the `pandas_datareader` package.

### 2. Getting the Data

We import the `pandas_datareader` below as well as `pandas` since we will be working with DataFrames.

```
In [ ]: import pandas as pd
import pandas_datareader as pdr # Access FRED
```

Now that we've imported the necessary package required to obtain the data from FRED, we specify the API key that we obtained after creating an account. It is quite an easy and

quick process to obtain an API key.

```
In [ ]: # extract api key: put your key in between the angle brackets < >
# myKey = "xxx"
fred_api_key = "<ENTER YOUR API KEY>"
```

We'll use this function created below called *get\_fred\_data*, which makes importing the data very simple. The input parameters are the list of economic indicator keywords and the date range of the data, i.e., start and end date.

```
In [ ]: # Using code from FRED API: Get US Economic Data using Python

def get_fred_data(param_list, start_date, end_date):
    df = pdr.DataReader(param_list, "fred", start_date, end_date)
    return df.reset_index()
```

Below is the list of the economic indicator keywords for the five predictors. These keywords are obtained from the FRED website. See below encapsulated in red in Figure 1 as an example.

### Figure 1: Keyword for Manufacturers' New Orders - Total Manufacturing.

A list of indicators used and considered is shown below in the next cell. We import the data and take a glimpse using the pandas *tail(10)* command to view the last 10 records. Note that we consider monthly data and take data from Jan. 2000 to Apr. 2022 inclusively.

```
In [ ]: # PCUOMFGOMFG :      Producer Price Index by Industry - Monthly
# BOGZ1FL073164003Q :      Interest Rates and Price Indexes; NYSE Composite Index, L
# CES0500000003 :          Average hourly earnings of all employees
# MANEMP:                  ALL Employees Manufacturing
# DGORDER:                 Durable goods New Orders
# AMTMNO:                  Manufacturers' New Orders: Total Manufacturing
# FEDFUNDS:                Fed fund rate. Lagging indicator

series = ["CES0500000003", "MANEMP", "PCUOMFGOMFG", "DGORDER", "AMTMNO"]
# get data for series
df = get_fred_data(param_list=series, start_date="2000-01-01", end_date="2022-05-01")
df.set_index("DATE", drop=True, inplace=True)
print(df.shape)
df.tail(10)
```

So now that we have our predictors, let's get the target variable, which is the adjusted close price for the Mini DJI futures. We get this data using the yahoo finance (*yfinance*) library. We import monthly data in keeping with the same timeframe as our predictors for the same time period. We have imported Open, High, Low, Close, Adj Close and Volume; however, we consider only the Adj Close that takes into account corporate actions.

```
In [ ]: import yfinance as yf

# Mini DJI Futures ticker is YM=F
data = yf.download(tickers="YM=F", start="2000-01-01", end="2022-05-03", interval="1h")
# Print data
print(data.shape)
data.tail(10)
```

We have our predictors and target variable in different DataFrames; however, they can be joined/merged using the common date index column. We do this using the pandas merge command. It is also necessary to look at missing values or Nulls in the dataset. This could be for unavailability of data on those dates. Fortunately, there are 39 rows missing for the Average hourly earnings of all employees (CES0500000003) column, which isn't too many. We adopt the simplest approach of dropping them since there aren't too many missing values. We are left with 166 rows.

```
In [ ]: # Make time zones non-timezone aware so as to allow the join
df.index = df.index.tz_localize(None)
data.index = data.index.tz_localize(None)
```

```
In [ ]: df2predict = pd.merge(df, data["Adj Close"], left_index=True, right_index=True)
print(df2predict.tail())
print(len(df2predict))
df2predict.isnull().sum()
df2predict = df2predict.dropna()

print("Print rows remaining after removed missing values {}".format(len(df2predict)))
```

We've done a lot of work to put this dataset together, so let's store it as a csv so we can easily read it in when we want to work with it. Specify the path to store it and the name; in my case, I called it `DJI_FuturesPredict.csv`. When we want to read the data in, we just use the pandas `read_csv` command as in the cell below.

```
In [ ]: # Store to csv
#path2copy = "../data/"
df2predict.to_csv("DJI_FuturesPredict.csv", index=True, index_label="Date")
```

```
In [ ]: # read in pre-stored data

df2predict = pd.read_csv("DJI_FuturesPredict.csv")
df2predict.set_index("Date", drop=True, inplace=True)

# Quick check that the data looks familiar.
df2predict.tail()
```

### 3. Data Analysis and Preparation

Neural Networks are sensitive to the scale of the features for accuracy as well as the speed of training. There is no single best way to scale the data as we could use percentage change, min max, or standard scaling, among other suggestions, and it depends on the problem. In this example, we use the min max scaling method from

`sklearn` . The formula for this is: 
$$x_{\text{scaled}} = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$
 where,

- $x_{\text{min}}$  = minimum feature value
- $x_{\text{max}}$  = maximum feature value

```
In [ ]: # Feature scaling
from sklearn.preprocessing import MinMaxScaler

# scale features
scaler = MinMaxScaler()
scale_model = scaler.fit(df2predict[series])
df2predict[series] = scale_model.transform(df2predict[series])
df2predict.tail()
```

We're interested in predicting the returns of the DJI Futures. As such, we transform the Adj Close price column to percentage returns instead.

```
In [ ]: # Use percent change instead of actual values
# If all columns are transformed to a % change.
# df2predict = df2predict.pct_change()

# % change for just the target column
df2predict["Adj Close"] = df2predict["Adj Close"].pct_change()

# Drop any missing values
df2predict.dropna(inplace=True)
# If a time shift is needed because of a lag.
# `df2predict['AdjClose_shift'] = df2predict['Adj Close'].shift(-1)`
# `df2predict.drop('Adj Close', axis=1, inplace=True)`

# Glimpse of data
df2predict.head()
```

It is a good idea to look at the relation of the predictors to the Adj Close price returns. We can do this visually since there aren't too many predictors considered. To avoid too much clutter in the plots and determine any time lag between the Adj Close returns and predictor, we choose a sample of points, say 3 years or 36 months. The visual below does not show strong correlations between the predictors and returns; however, the decrease in Feb. and Mar. of 2020 is experienced for most predictors as well as the returns.

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
```

```
In [ ]: target = "Adj Close"
n_pts = 36

for j in np.arange(len(series)):
    # Check the lag

    var = series[j]
    # Define Data
    # choose sample of points to avoid clutter in plot
    df2plot = df2predict.iloc[len(df2predict) - n_pts : len(df2predict),] # noC
    x = df2plot.index
```

```

data_1 = df2plot[var]
data_2 = df2plot[target]

# Create Plot

fig, ax1 = plt.subplots()

ax1.set_xlabel("Date")
ax1.set_ylabel(var, color="red")
ax1.plot(x, data_1, color="red")
plt.xticks(rotation=90) # Rotates X-Axis Ticks by 45-degrees
# Adding Twin Axes

ax2 = ax1.twinx()
ax2.plot(x, data_2, color="blue")

# Add Label

ax2.set_ylabel("Adj Close returns", color="blue")
ax2.tick_params(axis="y", color="blue")

# Show plot

plt.show()

```

With that, we are now ready to create our Neural Network using TensorFlow and Keras.

## 4. Keras and TensorFlow

Neural networks were initially made easily in Python thanks to the Keras library made by Francois Chollet. His book is also a great resource for neural networks and deep learning (Chollet 54); however, it is not open source.

Google created a library called TensorFlow (tf) that did the computations for neural networks efficiently, but it is not the most user-friendly.

Google later released TensorFlow 2 that integrated the Keras API directly and promoted this interface as the default or standard interface for deep learning development on the platform.

This is why we often see tf.keras used in deep learning code, which makes it much easier and faster to compile and run these models.

```

In [ ]: # Begin creating our model by importing TensorFlow
import tensorflow as tf

print(tf.__version__)

```

We now separate the data into a predictors dataset and target column into x and y respectively.

```

In [ ]: X = df2predict[series].values
y = df2predict["Adj Close"].values

```

We perform the necessary train/test split for our data required for predictive modeling. In our case the train/test split is 80/20 and the split is done in a time-ordered manner with recent data being the unseen dataset.

```
In [ ]: test_sz = 0.2
train_sz = int((1 - test_sz) * len(X))
X_train = X[:train_sz]
y_train = y[:train_sz]
X_test = X[train_sz:]
y_test = y[train_sz:]
len(X_train), len(X_test)
```

In the below cell, we will do a couple of things for the model build. We set a random seed to enable us to replicate this build at any time. We then call the various `tf.keras` objects to put our model together starting with *Sequential*. It is referred to as sequential because we are able to add layers to our NN one after another in sequence. In the simplest example for illustrative purposes, we have only one layer, but we will add onto it later. The `tf.keras.Dense` object should sound familiar from Module 4, Lesson 3, in that we are specifying that all edges connect to the subsequent nodes. The parameter of 1 in the `Dense(1)` object specifies that the output is a single continuous value, i.e., our predicted DJI Futures return.

The `model.compile` command is where we specify the loss function and optimizer. In our example below, we use the mean absolute error as our loss function to gauge how far away our predictions are from actuals. For a list of supported loss functions, see ("Keras Loss Functions").

The optimizer is SGD or stochastic gradient descent, but Adam is another popular option. The performance metric after each iteration is the mean absolute error, but we could also use mean square error. For a list of supported metrics, see ("Keras Metrics").

You'll notice there are specified arguments in the optimizer such as learning rate and momentum. The learning rate specifies the amount of change to the model relative to the change in error. Of course, we would naturally want our model to learn quickly; however, we should bear in mind that a large value for the learning rate could result in learning from a less than optimal set of weights too quickly, thus leading to non-convergence. This could result in an unstable learning process. A learning rate that is too small would increase the training time, possibly leading to the algorithm getting stuck. Typically, learning rate values between 0.01 to 1 are used. Momentum can speed up the training process to converge with fewer epochs. Values of momentum between 0.9 to 0.99 are often used.

Epochs are the number of times the algorithm will work through the training set. The more epochs, the more time the model needs to train. With limited computing power this needs to be given some thought as the time required could increase significantly. Another reason we can't just increase epochs to as high a number as possible is that this could also lead to over-fitting. We choose 10 here for a simple illustrative purpose but will increase it later to see the change in results. Note that small learning rates would require more epochs and vice versa.

The batch size is the sample size sent through at one time to be propagated through the network. The error gradient calculation is based on these samples that pass through. This would of course need to be smaller than the total number of observations in our training data. The stochastic gradient descent optimizer updates the weights based on the error gradient. Common sizes are in increments of 8. You don't want the batch size being too large a proportion of the number of training set observations. Refer to ("Batch Size Tutorial") for how to choose batch size.

It is worth noting that the batch size and epochs work with the entire training set (80% of observations) available. All observations of the training set are sampled in sizes of the batch size and fed through the algorithm at different times.

Now that we've specified the hyperparameters needed, we can compile and train the model with `model.fit`.

```
In [ ]: # Build the model
# tf.keras: The Keras API integrated into TensorFlow 2

tf.random.set_seed(42) # first we set random seed
model = tf.keras.Sequential([tf.keras.layers.Dense(1)]) # The output layer

# We compile the model specifying loss, and optimizer.
model.compile(
    loss=tf.keras.losses.mae, # Los function is MAE, mean absolute error.
    optimizer=tf.keras.optimizers.SGD(
        learning_rate=0.01, momentum=0.9
    ), # stochastic Gradient descent Optimizer
    metrics=["mae"],
) # performance metric is MAE

model.fit(X_train, y_train, epochs=10, batch_size=8) # epoch and batch size spe
```

After training our model, we can look at performance. We use the mean absolute error and mean square error since we are dealing with a continuous target variable. We see that both these metrics are below 5% and 0.3%, respectively.

```
In [ ]: # performance
preds = model.predict(X_test)
mae = tf.metrics.mean_absolute_error(y_true=y_test, y_pred=preds.squeeze()).numpy
mse = tf.metrics.mean_squared_error(y_true=y_test, y_pred=preds.squeeze()).numpy
mae, mse
```

Let's see if we can improve on this result with more layers and different choices of the hyperparameters, such as increasing the number of epochs. We also specify an activation function, namely the *sigmoid* function in this example. We also use the Adam optimizer function instead of the SGD. The SGD optimizer is sensitive to the learning rate, and although it can take many iterations, it can still achieve convergence relatively quickly. Adam is a popular optimizer and is used quite effectively in deep learning algorithms. Unlike SGD, Adam does not use a fixed learning rate as it adapts over iterations.

We see that both metrics, MAE and MSE have decreased with 2 additional layers. Adding more layers ventures into the space of Deep Learning and helps when data is non-linear.

Remember that the algorithm becomes susceptible to the vanishing gradient problem as more layers are added, especially when the activation function is the sigmoid function.

```
In [ ]: # Improve our model. More epochs, added extra layers

tf.random.set_seed(42)
model_1 = tf.keras.Sequential(
    [
        tf.keras.layers.Dense(
            8, activation="sigmoid", input_shape=(X_train.shape[1],)
        ), # added extra layer
        tf.keras.layers.Dense(4, activation="sigmoid"), # added extra layer
        tf.keras.layers.Dense(1),
    ]
)
model_1.compile(
    loss=tf.keras.losses.mae, optimizer=tf.keras.optimizers.Adam(), metrics=["ma
")
model_1.fit(X_train, y_train, epochs=100, batch_size=8, verbose=0)
```

```
In [ ]: # performance
preds = model_1.predict(X_test)
mae = tf.metrics.mean_absolute_error(y_true=y_test, y_pred=preds.squeeze()).numpy
mse = tf.metrics.mean_squared_error(y_true=y_test, y_pred=preds.squeeze()).numpy
mae, mse
```

## 5. Conclusion

In this lesson, we applied a neural network to predicting the monthly returns of the DJI Futures. We used economic indicators from FRED as predictors and explored ways to import and format this data for a predictive analytics scenario. In setting up the neural network, we discussed several hyperparameters required to compile the network and added more layers in our last attempt. This addition of more layers is necessary when creating a deep learning model.

### References

- Chollet, Francois. *Deep Learning with Python*. Manning, 2017.
  - "Keras Loss Functions." Keras Loss Functions List, [www.tensorflow.org/api\\_docs/python/tf/keras/losses](http://www.tensorflow.org/api_docs/python/tf/keras/losses). Accessed 18 June 2022.
  - "Keras Metrics." Keras Metrics List, [www.tensorflow.org/api\\_docs/python/tf/keras/metrics](http://www.tensorflow.org/api_docs/python/tf/keras/metrics). Accessed 18 June 2022.
  - "Batch Size Tutorial." Batch Size Tutorial, [www.machinelearningmastery.com/how-to-control-the-speed-and-stability-of-training-neural-networks-with-gradient-descent-batch-size](http://www.machinelearningmastery.com/how-to-control-the-speed-and-stability-of-training-neural-networks-with-gradient-descent-batch-size). Accessed 18 June 2022.
-



# Multiclass Optimal Classification Trees with SVM-splits

Víctor Blanco<sup>†</sup>, Alberto Japón<sup>‡</sup> and Justo Puerto<sup>‡</sup>

<sup>†</sup>Institute of Mathematics (IMAG), Universidad de Granada

<sup>‡</sup>Institute of Mathematics (IMUS), Universidad de Sevilla

vblanco@ugr.es, ajapon1@us.es, puerto@us.es

**ABSTRACT.** In this paper we present a novel mathematical optimization-based methodology to construct tree-shaped classification rules for multiclass instances. Our approach consists of building Classification Trees in which, except for the leaf nodes, the labels are temporarily left out and grouped into two classes by means of a SVM separating hyperplane. We provide a Mixed Integer Non Linear Programming formulation for the problem and report the results of an extended battery of computational experiments to assess the performance of our proposal with respect to other benchmarking classification methods.

## 1. INTRODUCTION

Interpretability is a crucial requisite demanded to machine learning methods provoked by the tremendous amount of methodologies that have arisen in the last decade [21]. It is expected that the model that results when applying a machine learning methodology using a training sample, apart from being able to adequately predict the behaviour of out-of-sample observations, can be interpreted. Different tools have been applied to derive *interpretable* machine learning methods. One of the most popular strategies to simplify the obtained models is *feature selection*, in which a reduced set of attributes is to be selected without losing quality in the predictions. Reducing the number of parameters to analyze, the models can be easier to understand, yielding higher descriptive accuracy. One could also consider models that can be modulated, in the sense that a great proportion of its prediction-making process can be interpreted independently. This is the case of generalized linear models [29]. Other methods incorporate interpretability as a synonym of being able to be reproduced by humans in its entire construction [15, 34]. This is the case of Decision Trees with small depth which can be visualized and interpreted easily by users even

not familiar with the tools behind their construction. We adopt a tree-based methodology through this paper.

Among the wide variety of strands derived under the lens of Machine Learning, classification is one that has attracted a lot of attention because of its applicability in many different fields [4, 28, 31, 36, 41]. Classification methodologies aim to adequately predict the class of new observations provided that a given sample has been used to construct the *classification rule*. The role of Mathematical Programming in the construction of classification models has been widely recognized, and some of the most popular methods to derive classification rules are based on solving optimization problems [13, 17, 16, 11, 27]. Moreover, Mathematical Programming has also been proven to be a flexible and accurate tool when requiring interpretability to the obtained models [5, 6, 12, 25].

However, most of the optimization tools derived to construct classifiers assume instances with only two classes. In this paper, we provide a novel classification method in which the instances are allowed to be classified into two or more classes. The method is constructed using one of the most interpretable classification method, Classification Trees, but combined with Support Vector Machines, which provide highly predictive models.

We have developed a Mathematical Programming model that allows to construct an Optimal Classification Tree for a given training sample, in which each split is generated by means of a SVM-based hyperplane. When building the tree, the labels of the observations are ignored in the branch nodes, and they are only accounted for in the leaf nodes where misclassification errors are considered. The classification tree is constructed to minimize the complexity of the tree (assuring interpretability) and also the misclassification risk (assuring predictive power).

**1.1. Related Works.** Several machine learning methodologies have been proposed in the literature in order to construct highly predictive classification rules. The most popular ones are based on Deep Learning mechanisms [1],  $k$ -Nearest Neighborhoods [18, 42], Naïve Bayes [35], Classification Trees (CT) [15, 23] and Support Vector Machines (SVM) [17]. Among them, CT and SVM, which are, by nature, optimization-based methodologies, apart from producing highly predictive classifiers have been proven to be very flexible tools since both allow the incorporation of different elements (through the adequate optimization models by means of constraints and objective functions) to be adapted to different situations, as Feature Selection [27, 5, 6, 30], accuracy requirements [7, 24] or dealing with unbalanced or noisy instances [22, 10, 14], amongst others.

Support Vector Machines were originally introduced by Cortes and Vapnik [17] as a binary classification tool that builds the decision rule by means of a separating hyperplane with large separation between the two classes. This hyperplane is

obtained by solving a convex quadratic optimization problem, in which the goal is to separate data by their two differentiated classes, maximizing the margin between them and minimizing the misclassification errors. Duality properties of this optimization problem allow one to extend the methodology to find nonlinear separators by means of kernels. Classification Trees were firstly introduced by Breiman et. al [15], and the decision rule is based on a hierarchical relation among a set of nodes which is used to define paths that lead observations from the root node (highest node in the hierarchical relation), to some of the leaves in which a class is assigned to the data. These paths are obtained according to different optimization criteria over the predictor variables of the training sample. The decision rule comes up naturally, the classes predicted for new observations are the ones assigned to the terminal nodes in which observations fall in. Clearly, the classification rules derived from CTs are easily interpretable by means of the splits that are constructed at the tree nodes. In [15], a greedy heuristic procedure, the so-called CART approach, is presented to construct CTs. Each level of the tree is sequentially constructed: starting at the root node and using the whole training sample, the method minimizes an impurity measure function obtaining as a result a split that divides the sample into two disjoint sets which determine the two descendant nodes. This process is repeated until a given termination criteria is reached (minimum number of observations belonging to a leaf, maximum depth of the tree, or minimum percentage of observations of the same class on a leaf, amongst others). In this approach, the tree grows following a top-down greedy approach, an idea that is also shared in other popular decision tree methods like C4.5 [40] or ID3 [39]. The advantage of these methods is that the decision rule can be obtained rather quickly even for large training samples, since the whole process relies on solving manageable problems at each node. Nevertheless, these types of heuristic approaches may not obtain the *optimal* classification tree, since they look for the best split locally at each node, not taking into account the splits that will come afterwards. Thus, these local branches may not capture the proper structure of the data, leading to misclassification errors in out-of-sample observations. Furthermore, the solutions provided by these methods can result into very deep (complex) trees, resulting in overfitting and, at times, loosing interpretability of the classification rule. This difficulty is usually overcome by pruning the tree as it is being constructed by comparing the gain on the impurity measure reduction with respect to the complexity cost of the tree.

The recent advances on modeling and solving difficult Optimization problems together with the flexibility and adaptability of these models have motivated the use of optimization tools to construct supervised classification methods with a great success [9, 16]). In particular, recently, Bertsimas and Dunn [8] introduced

the notion of *Optimal Classification Trees* (OCT) by approaching Classification and Regression Trees under optimization lens, providing a Mixed Integer Linear Programming formulation for its optimal construction. Moreover, the authors proved that this model can be solved for reasonable size datasets, and equally important, that for many different real datasets, significant improvements in accuracy with respect to CART can be obtained. In contrast to the standard CART approach, OCT builds the tree by solving a single optimization problem taking into account (in the objective function) the complexity of the tree, avoiding post pruning processes. Moreover, every split is directly applied in order to minimize the misclassification errors on the terminal nodes, and hence, OCTs are more likely to capture the hidden patterns of the data.

While SVM were initially designed to deal only with bi-class instances, some extensions have been proposed in the literature for multiclass classification. The most popular multiclass SVM-based approaches are One-Versus-All (OVA) and One-Versus-One (OVO). The former, namely OVA, computes, for each class  $r \in \{1, \dots, k\}$ , a binary SVM classifier labeling the observations as 1, if the observation is in the class  $r$ , and  $-1$  otherwise. The process is repeated for all classes ( $k$  times), and then each observation is classified into the class whose constructed hyperplane is the furthest from it in the positive halfspace. In the OVO approach, classes are separated with  $\binom{k}{2}$  hyperplanes using one hyperplane for each pair of classes, and the decision rule comes from a voting strategy in which the most represented class among votes becomes the class predicted. OVA and OVO inherit most of the good properties of binary SVM. In spite of that, they are not able to correctly classify datasets where separated clouds of observations may belong to the same class (and thus are given the same label) when a linear kernel is used. Another popular method is the directed acyclic graph SVM, DAGSVM [11]. In this technique, although the decision rule involves the same hyperplanes built with the OVO approach, it is not given by a unique voting strategy but for a sequential number of votings in which the most unlikely class is removed until only one class remains. In addition, apart from OVA and OVO, there are some other methods based on decomposing the original multiclass problem into several binary classification ones. In particular, in [2] and [20], this decomposition is based on the construction of a coding matrix that determines the pairs of classes that will be used to build the separating hyperplanes. Alternatively, other methods such as CS ([19]), WW ([44]) or LLW ([33]), do not address the classification problem sequentially but as a whole considering all the classes within the same optimization model. Obviously, this seems to be the correct approach. In particular, in WW,  $k$  hyperplanes are used to separate the  $k$  classes, each hyperplane separating one class from the others, using  $k - 1$  misclassification errors for each observation. The same separating idea, is applied

in CS but reducing the number of misclassification errors for each observation to a unique value. In LLW, a different error measure is proposed to cast the Bayes classification rule into the SVM problem implying theoretical statistical properties in the obtained classifier. These properties cannot be ensured in WW or CS.

We can also find a quadratic extension based on LLW proposed by [26]. In [43], the authors propose a multiclass SVM-based approach, *GenSVM*, in which the classification boundaries for a problem with  $k$  classes are obtained in a  $(k - 1)$ -dimensional space using a simplex encoding. Some of these methods have become popular and are implemented in most software packages in machine learning as `e1071` [37], `scikit-learn` [38] or [32]. Finally, in the recent work [11] the authors propose an alternative approach to handle multiclass classification extending the paradigm of binary SVM classifiers by constraining a polyhedral partition of the feature space and an assignment of classes to the *cells* of the partition, by maximizing the separation between classes and minimizing two intuitive misclassification errors.

**1.2. Contributions.** In this paper, we propose a novel approach to construct Classification Trees for multiclass instances by means of a mathematical programming model. Our method is based on two main ingredients: (1) An optimal binary classification tree (with oblique cuts) is constructed in the sense of [8], in which the splits and pruned nodes are determined in terms of the misclassification errors at the leaf nodes; (2) The splits generating the branches of the tree are built by means of binary SVM-based hyperplanes separating fictitious classes (which are also decided by the model), i.e., maximizing separation between classes and minimizing the distance-based misclassification errors.

Our specific contributions include:

- (1) Deriving an interpretable multiclass classification rule which combines two of the most powerful tools in supervised classification, namely OCT and SVM.
- (2) The classifier is constructed using a mathematical programming model that can be formulated as a Mixed Integer Second Order Cone Programming problem. The classifier is simple to apply and interpretable.
- (3) Several valid inequalities are presented for the formulation that allow one to strengthen the model and to solve larger size instances in smaller CPU times.
- (4) An extensive battery of computational experiments on realistic datasets from UCI is reported showing that our approach outperforms other decision tree-based methodologies as CART, OCT and OCT-H.

**1.3. Paper structure.** Section 2 is devoted to fix the notation and to recall the tools that are used to derive our method. In Section 3 we detail the main ingredients of our approach and illustrate its performance on a toy example. The mathematical programming model that allows us to construct the classifier is given in Section 4, where we include all the elements involved in the model: parameters, variables, objective function and constraints. In Section 5 we report the results of our experiments to assess the performance of our method compared with other tree-shaped classifiers. Finally, Section 6 is devoted to draw some conclusions and future research lines on the topic.

## 2. PRELIMINARIES

This section is devoted to introduce the problem under study and to fix the notation used through this paper. We also recall the main tools involved in our proposed approach namely, Support Vector Machines and Optimal Classification Trees. These methods are adequately combined to develop a new method, called Multiclass Optimal Classification Trees with Support Vector Machines based splits (MOCTSVM).

We are given a training sample,  $\mathcal{X} = \{(x_1, y_1), \dots, (x_n, y_n)\} \subseteq \mathbb{R}^p \times \{1, \dots, K\}$ , which comes up as the result of measuring  $p$  features over a set of  $n$  observations  $(x_1, \dots, x_n)$  as well as a label in  $\{1, \dots, K\}$  for each of them  $(y_1, \dots, y_n)$ . The goal of a classification method is to build a decision rule so as to accurately assign labels ( $y$ ) to data ( $x$ ) based on the behaviour of the given training sample  $\mathcal{X}$ .

The first ingredient that we use in our approach is the Support Vector Machine method. SVM is one of the most popular optimization-based methods to design a classification rule in which only two classes are involved, usually referred as the positive ( $y = +1$ ) and the negative class ( $y = -1$ ). The goal of linear SVM is to construct a hyperplane separating the two classes by maximizing their separation and simultaneously minimizing the misclassification and margin violation errors. Linear SVM can be formulated as the following convex optimization problem:

$$\begin{aligned} \min \quad & \frac{1}{2} \|\omega\|_2^2 + c \sum_{i \in N} e_i \\ \text{s.t.} \quad & y_i(\omega' x_i + \omega_0) \geq 1 - e_i, & \forall i \in N, \\ & \omega \in \mathbb{R}^p, \omega_0 \in \mathbb{R}, \\ & e_i \in \mathbb{R}_+, & \forall i \in N. \end{aligned}$$

where  $c$  is the regularization parameter that states the trade-off between training errors and model complexity (margin),  $\omega'$  is the transpose of the vector  $\omega$  and  $\|\cdot\|_2$  is the Euclidean norm in  $\mathbb{R}^p$  (other norms can also be considered but still

keeping similar structural properties of the optimization problem [13]). Note that with this approach, the positive (resp. negative) class will tend to lie on the positive (resp. negative) half space induced by the hyperplane  $\mathcal{H} = \{z \in \mathbb{R}^p : \omega'z + \omega_0 = 0\}$ . On the other hand, the popularity of SVM is mostly due to the so call kernel trick. This allows one to project the data onto a higher dimensional space in which a linear separation is performed in a most accurate way with no need of knowing such a space, but just knowing the form of its inner products, and maintaining the computational complexity of the optimization problem (see [17] for further details).

The second method that we combine in our approach is Classification Trees. CTs are a family of classification methods based on a hierarchical relationship among a set of nodes. These methods allow one to create a partition of the feature space by means of hyperplanes that are sequentially built. CT starts on a node containing the whole sample, that is called the root node, in which the first split is applied. When applying a split on a node (by means of a hyperplane separating the observations, two new branches are created leading to two new nodes, which are referred to as its child nodes. The nodes are usually distinguished into two groups: branch nodes, that are nodes in which a split is applied, and on the other hand the leaf nodes, which are the terminal nodes of the tree. Given a branch node and a hyperplane split in such a node, their branches (left and right) are defined as each of the two halfspaces defined by the hyperplane. The final goal of CT is to construct branches in order to obtain leaf nodes as pure as possible with respect to the classes. In this way, the classification rule for a given observation consists of assigning it to the most popular class of the leaf where it belongs to.

There is a vast amount of literature on CTs since they provide an easy interpretable classification rule. One of the most popular methods to construct CT is known as CART, introduced in [15]. CART is a greedy heuristic approach that starts at the root node looking for the split in a single feature that minimizes a given impurity function, creating two new nodes. The same procedure is sequentially applied until a stop criteria is reached (maximal depth of the tree, proportion of observations of a single class in a node, etc). The main advantage of CART is its low computational cost, since nowadays very deep trees can be obtained within a few seconds. However, CART does not guarantee the *optimality* of the classification tree, in the sense that more accurate trees could be obtained if instead of locally constructing the branches one looks at the final configuration of the leaf nodes. For instance, in Figure 1(left) we show a CT constructed by CART for a biclass problem with maximal depth 2. We draw the classification tree, and also in the top right corner, the partition of the feature space (in this case  $\mathbb{R}^2$ ). As can be observed, the obtained classification is not

perfect (not all leaf nodes are composed by pure classes) while in this case is not difficult to construct a CT with no classification errors. This situation is caused by the myopic construction done by the CART approach that, at each node only cares on better classification at their children, but not at the final leaf nodes, while subsequent branching decisions clearly affect the overall shape of the tree.

Motivated by this drawback of CART, in [8], the authors propose an approach to build an Optimal Classification Tree (OCT) by solving a single Mathematical Programming problem in which not only single-variable splits are possible but *oblique* splits involving more than one predictive variable (by means of general hyperplanes in the feature space) can be constructed. In Figure 1(right) we show a solution provided by OCT with hyperplanes (OCT-H) for the same example. One can observe that when splitting the root node (orange branches) a good local split is not obtained (the nodes contain half of the observations in different classes), however, when adding the other two splits, the final leaves only have observations of the same class, resulting in a perfect classification rule for the training sample.

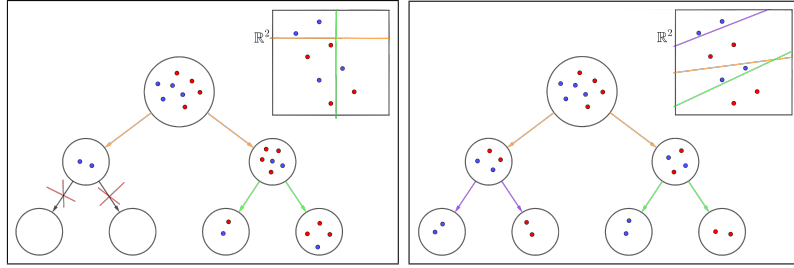


FIGURE 1. Example of a CT obtained with CART (left) and OCT-H (right) approaches for the same instance.

Both approaches, OCTs and SVMs can be combined in order to construct classification trees in which the classes separated by the hyperplanes determined in the CT are *maximally* separated, in the sense of the SVM approach. This idea is not new and has been proven to outperform standard optimal decision trees methods amongst many different biclass classification problems, as for instance, in [12] where the OCTSVM method is proposed. In Figure 2 we show how one could construct OCTs with larger separations between the classes using OCTSVM but still with the same 100% accuracy in the training sample as in OCT-H, but more protected to misclassification in out-sample observations.

Nevertheless, as far as we know, the combination of OCT and SVM has only been analyzed for biclass instances. The extension of this method to multiclass settings (more than two classes) is not trivial, since one could construct more complex trees or use a multiclass SVM-based methodology (see e.g. [19, 44, 33]).

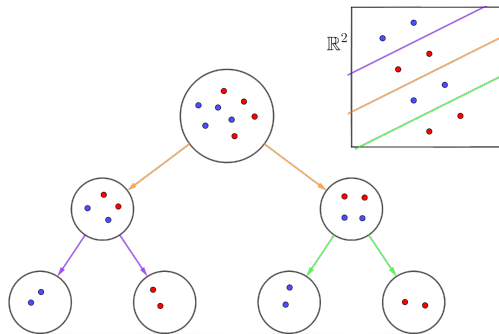


FIGURE 2. Example of a CT obtained with OCTSVM.

However, these adaptations of the classical SVM method have been proved to fail in real-world instances (see [11]). In the rest of the paper we describe a novel methodology to construct accurate multiclass tree-shaped classifiers based on a different idea: constructing CTs with splits induced by bi-class SVM separators in which the classes of the observations at each one of the branch nodes are determined by the model, but adequately chosen to provide small classification errors at the leaf nodes. The details of the approach are given in the next section.

### 3. MULTICLASS OCT WITH SVM SPLITS

In this section we describe the method that we propose to construct classification rules for multiclass instances, in particular Classification Trees in which splits are generated based on the SVM paradigm.

As already mentioned, our method is based on constructing OCT with SVM splits, but where the classes of the observations are momentarily ignored and only accounted for at the leaf nodes. In order to illustrate the idea under our method, in Figure 3 we show a toy instance with a set of points with four different classes (blue, red, orange and green).

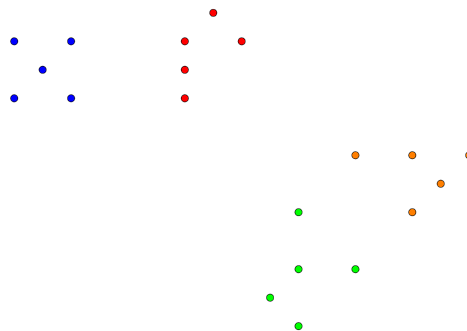


FIGURE 3. Instance for a 4-class problem.

First, at the root node (the one in which all the observations are involved), our method constructs a SVM separating hyperplane for two fictitious classes (which have to be also decided). A possible separation could be the one shown in Figure 4, in which the training dataset has been classified into two classes (black and white). This separation allows one to generate two child nodes, the

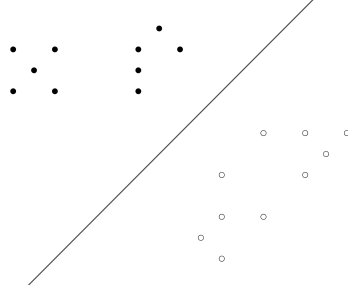


FIGURE 4. Root split on the 4-class classification problem

black and the white nodes. At each of these nodes, the same idea is applied until the leaf nodes are reached. In Figure 5 we show the final partition of the feature space according to this procedure.

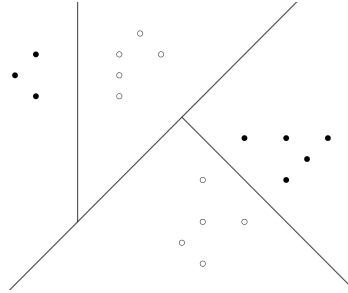


FIGURE 5. Child node splits on the 4-class classification problem higheleted as the fictitious classes decided in our model.

Clearly, ignoring the original classes of the training sample in the whole process would result in senseless trees, unless one accounts for the goodness in the classification rule in the training sample at the leaf nodes. Thus, at the final leaf nodes, the original labels are recovered and the classification is performed according to the generated hyperplanes. The final result of this tree is shown in Figure 6 where one can check that the constructed tree achieves a perfect classification of the training sample.

Once the tree is constructed with this strategy, the decision rule comes up naturally as it is usually done in decision trees methods, that is, out of sample observations will follow a path on the tree according to the splits and they will be assigned to the class of the leaf where they lie in (the most represented class

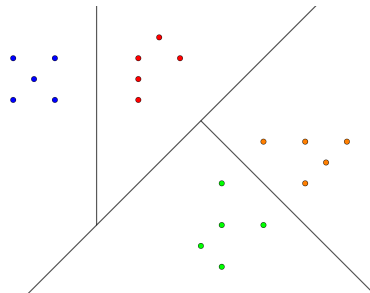


FIGURE 6. Child node splits on the 4-class classification problem with their original labels (colors).

of the leaf over the training set). In case a branch is pruned when building the tree, observations will be assigned to the most represented class of the node where the prune took place.

#### 4. MATHEMATICAL PROGRAMMING FORMULATION FOR MOCTSVM

In this section we derive a Mixed Integer Non Linear Programming formulation for the MOCTSVM method described in the previous section.

We assume to be given a training sample  $\mathcal{X} = \{(x_1, y_1), \dots, (x_n, y_n)\} \subseteq \mathbb{R}^p \times \{1, \dots, K\}$ . We denote by  $N = \{1, \dots, n\}$  the index set for the observations in the training sample. We also consider the binary representation of the labels  $y$  as:

$$Y_{ik} = \begin{cases} 1 & \text{if } y_i = k, \\ 0 & \text{otherwise,} \end{cases} \quad \text{for all } i \in N, k = 1, \dots, K.$$

Moreover, without loss of generality we will assume the features to be normalized, i.e.,  $x_1, \dots, x_n \in [0, 1]^p$ .

We will construct decision trees with a fixed maximum depth  $D$ . Thus, the classification tree is formed by at most  $T = 2^{D+1} - 1$  nodes. We denote by  $\tau = \{1, \dots, T\}$  the index set for the tree nodes, where node 1 is the root node and nodes  $2^D, \dots, 2^{D+1} - 1$  are the leaf nodes.

For any node  $t \in \tau \setminus \{1\}$ , we denote by  $p(t)$  its (unique) parent node. The tree nodes can be classified in two sets: branching and leaf nodes. The branching nodes, that we denote by  $\tau_b$ , will be those in which the splits are applied. In contrast, in the leaf nodes, denoted by  $\tau_l$ , no splits are applied but is where predictions take place. The branching nodes can be also classified into two sets:  $\tau_{bl}$  and  $\tau_{br}$  depending on whether they follow the left or the right branch on the path from their parent nodes, respectively.  $\tau_{bl}$  nodes are indexed with even numbers meanwhile  $\tau_{br}$  nodes are indexed with odd numbers.

We define a level as a set of nodes which have the same depth within the tree. The number of levels in the tree to be built is  $D + 1$  since the root node is assumed as the zero-level. Let  $U = \{u_0, \dots, u_D\}$  be the set of levels of the tree,

where each  $u_s \in U$  is the set of nodes at level  $s$ , for  $s = 0, \dots, D$ . With this notation, the root node is  $u_0$  while  $u_D$  represent the set of leaf nodes.

In Figure 7 we show the above mentioned elements in a 3-depth tree.

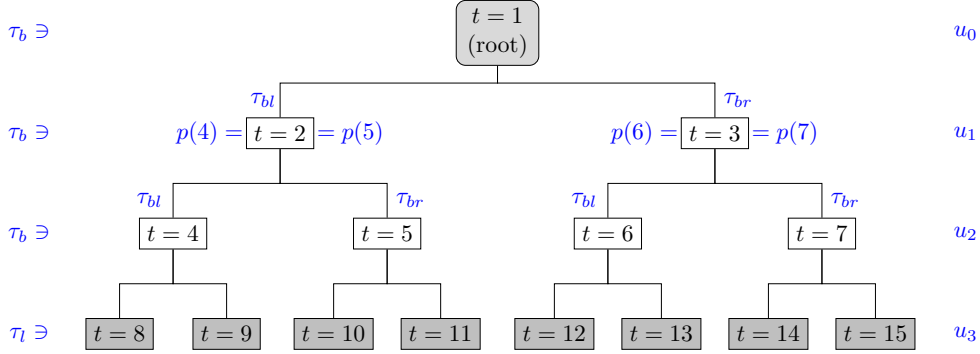


FIGURE 7. Elements in a depth  $D = 3$  tree.

Apart from the information about the topological structure of the tree, we also consider three regularization parameters that have to be calibrated in the validation process that allow us to find a trade-off between the different goals that we combine in our model: margin violation and classification errors of the separating splitting hyperplanes, correct classification at the leaf nodes and complexity of the tree. These parameters are the following:

$c_1$ : unit misclassification cost at the leaf nodes.

$c_2$ : unit distance based misclassification errors for SVM splits.

$c_3$ : unit cost for each splitting hyperplane introduced in the tree.

The complete list of index sets and parameters used in our model are summarized in Table 1.

**4.1. Variables.** Our model uses a set of decision and auxiliary variables that are described in Table 2. We use both binary and continuous decision variables to model the MOCTSVM. The binary variables allow us to decide the allocation of observations to the decision tree nodes, or to decide whether a node is splitted or not in the tree. The continuous variables allow us to determine the coefficients of the splitting hyperplanes or the misclassification errors (both in the SVM separations or at the leaf nodes). We also use auxiliary binary and integer variables that are useful to model adequately the problem.

In Figure 8 we illustrate the use of these variables in a feasible solution of a toy instance with three classes (red, blue and green).

|                           |                                                                                 |
|---------------------------|---------------------------------------------------------------------------------|
| $N = \{1, \dots, n\}$     | Index set for the observations in the training sample.                          |
| $D$                       | Maximal depth of the tree.                                                      |
| $T = 2^{D+1}$             | maximal number of nodes in a $D$ -depth tree.                                   |
| $\tau = \{1, \dots, T\}$  | Index set for the set of nodes of the tree.                                     |
| $p(t)$                    | Parent of node $t$ , for $t \in \tau \setminus \{1\}$ .                         |
| $\tau_b \in \tau$         | Branching nodes of the tree.                                                    |
| $\tau_l$                  | Leaf nodes of the tree.                                                         |
| $\tau_{bl} \in \tau_b$    | Nodes that follow the left branch on the path from their parent nodes.          |
| $\tau_{br} \in \tau_b$    | Nodes whose right branch has been followed on the path from their parent nodes. |
| $u_s$                     | Nodes at level $s$ of the tree, for $s = 0, \dots, D$ .                         |
| $U = \{u_0, \dots, u_d\}$ | Sets of levels of the tree.                                                     |
| $c_1$                     | Unit misclassification cost.                                                    |
| $c_2$                     | Unit distance based missclassification errors for SVM splits.                   |
| $c_3$                     | Unit cost for splitting hyperplanes.                                            |

TABLE 1. Index sets and parameters used in our model.

| Continuous Decision Variables |                                                                                                                                  |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| $\omega_t \in \mathbb{R}^p$   | Coefficients of the separating hyperplane of node $t$ .                                                                          |
| $\omega_{t0} \in \mathbb{R}$  | Intercept of the separating hyperplane of node $t$ .                                                                             |
| $e_{it} \in \mathbb{R}_+$     | Misclassification error of observation $i$ at node $t$ .                                                                         |
| $\delta \in \mathbb{R}_+$     | Inverse of the minimum margin between splitting hyperplanes.                                                                     |
| Binary Decision Variables     |                                                                                                                                  |
| $z_{it} \in \{0, 1\}$         | Is one if observation $i$ belongs to node $t$ and zero otherwise.                                                                |
| $d_t \in \{0, 1\}$            | Is one if a split is applied at node $t$ and zero otherwise.                                                                     |
| Auxiliary Variables           |                                                                                                                                  |
| $L_t \in \mathbb{Z}_+$        | Number of misclassified observations at leaf node $t$ .                                                                          |
| $\alpha_{it} \in \{0, 1\}$    | Is one if observation $i$ belongs to the reference fictitious class in node $t$ and zero otherwise.                              |
| $h_{it} \in \{0, 1\}$         | Is one if observation $i$ is in node $t$ and lies on the positive half space of the hyperplane of node $t$ , and zero otherwise. |
| $v_t \in \{0, 1\}$            | Is one if not all observations in node $t$ lie on the positive half space of the hyperplane in node $t$ and zero otherwise.      |
| $q_{kt} \in \{0, 1\}$         | Is one if class $k$ is the most represented one in leaf node $t$ and zero otherwise.                                             |

TABLE 2. Summary of the variables used in our model.

The whole set of training observation is considered at the root node (node  $t = 1$ ). There, the original labels are ignored and to determine the *fictitious* class of each observation a SVM-based hyperplane is constructed. Such a hyperplane

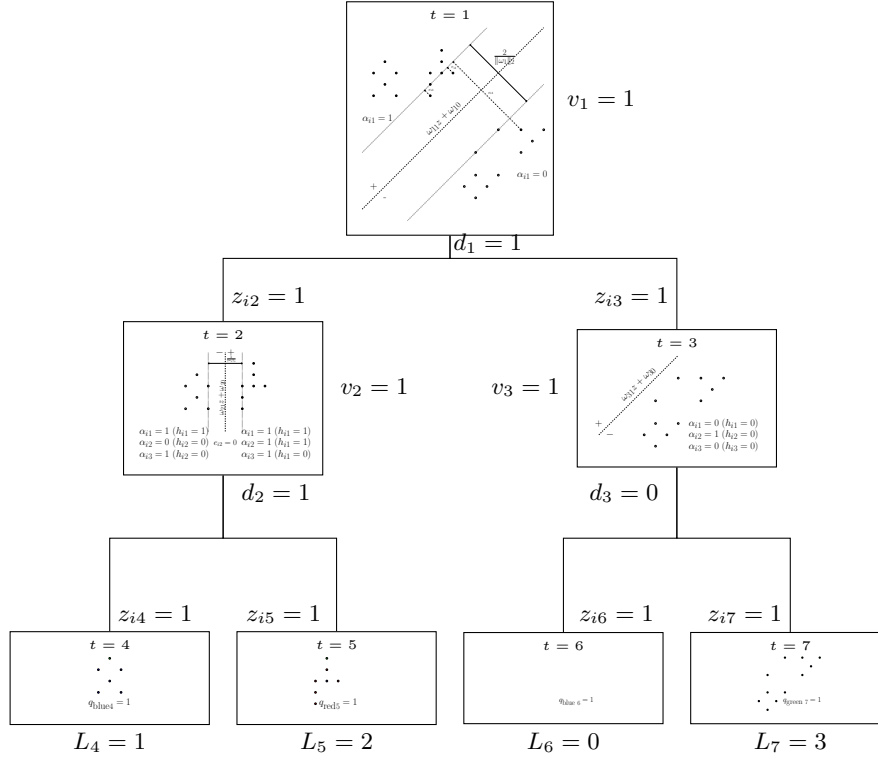


FIGURE 8. Illustration of the sets of variables used in our model in a toy example.

is defined by the coefficients  $\omega_1 \in \mathbb{R}^p$  and  $\omega_{10} \in \mathbb{R}$  (hyperplane/line drawn with a dotted line in the picture) and it induces a margin separation  $(\frac{2}{\|\omega_1\|_2})$  and misclassification errors  $e_{i1}$ . In the feasible solution drawn in the figure, only three observations induce positive errors (those that are classified either in the margin area or in the opposite side of the hyperplane). Such a hyperplane also determines the splitting rule for the definition of the children of that node. Since the node is split ( $d_1 = 1$ ), the observations that belongs to the *positive side* of the hyperplane are assigned to the left node (node  $t = 2$ ) while those in the negative side are assigned to the right node (node  $t = 3$ ) through the  $z$ -variables. At node  $t = 2$ , the same scheme is applied, that is, the hyperplane defined by  $\omega_2$  is constructed, inducing SVM-based margin and errors and since  $d_2 = 1$ , also the splitting rule applies to define nodes  $t = 4$  and  $t = 5$ . At node  $t = 2$ , one must control the observations in that node to quantify the misclassifying errors,  $e_{i2}$ , only for those observations in the objective function. Specifically, we only account for these errors for the observations that belong to the node ( $z_{i2} = 1$ ) and either belong to the positive ( $\alpha_{i2} = 1$ ) or the negative ( $\alpha_{i2} = 0$ ) side of the hyperplane. Also, in order to control the complexity of the tree, the  $h$ -variables are used to know whether an observation belongs to the node and to the positive

side of the SVM-hyperplane. If all observations in a node belong to the positive side of the hyperplane, the variable  $v$  assumes the value 0. Otherwise, in case  $v$  takes value 1, two situations are possible: 1) there are observations in both sides of the hyperplane (as in node  $t = 2$ ) inducing a new split ( $d_2 = 1$ ), and 2) all observations belong to the negative side (as in node  $t = 3$ ) determining that the tree is pruned at that node ( $d_3 = 0$ ).

Concerning the leaf nodes, node  $t = 2$  is split into nodes  $t = 4$  and  $t = 5$  and node  $t = 3$ , which was decided to be no longer split, is fictitiously split in two leaf nodes, although one of them is empty and the other one receives all the observations of the parent node (node  $t = 3$ ). The allocation of any leaf node  $\tau_l$  to a class is done through the  $q$ -variables (to the most popular class in the node or arbitrarily in case the node has no observations) and the number of misclassified observations is accounted for by the  $L$ -variables.

**4.2. Objective Function.** As already mentioned, our method aims to construct classification trees with small misclassification errors at the leaf nodes, but at the same time with maximal separation between the classes with the SVM-based hyperplanes and minimum distance based errors.

Using the variables described in the previous section, the four terms that are included in the objective functions are the following:

**Margins of the splitting hyperplanes::** The separating hyperplane of branching node  $t \in \tau_b$  has margin  $\frac{2}{\|\omega_t\|_2}$ . Thus, our method aims to maximize the minimum of these margins. This is equivalent to minimize the maximum among the inverse margins  $\{\frac{1}{2}\|\omega_t\|_2^2 : t \in \tau_b\}$  which is represented by the auxiliary variable  $\delta$ .

**Misclassification Errors at the leaf nodes::** Variable  $L_t$  accounts for the number of misclassified observations in leaf node  $t$ , i.e., the number of observations that do not belong to the most represented class in that leaf node. These variables allow us to count the overall number of misclassified observations in the training sample. Therefore, the amount to be minimized by the model is given by the following sum:

$$c_1 \sum_{t \in \tau_l} L_t$$

**Distance-based Errors at branching nodes::** Each time a split is added to the tree, a SVM-based hyperplane in which the labels are assigned based on the global convenience of for the overall tree is incorporated. Thus, we measure, at each branching node in  $\tau_b$ , the distance-based errors incurred by the SVM classifier at that split. This amount is measured

by the  $e_{it}$  variables and is incorporated to the model through the sum:

$$c_2 \sum_{i \in N} \sum_{t \in \tau_b} e_{it}$$

**Complexity of the tree:** The simplicity of the resulting tree is measured by the number of splits that are done in its construction. Since the  $d_t$  variable tells us whether node  $t$  is split or not, this term is accounted for in our model as:

$$c_3 \sum_{t \in \tau_b} d_t$$

Summarizing, the overall objective function of our model is:

$$\min \delta + c_1 \sum_{t \in \tau_l} L_t + c_2 \sum_{i \in N} \sum_{t \in \tau_b} e_{it} + c_3 \sum_{t \in \tau_b} d_t. \quad (\text{OBJ})$$

Note that the coefficients  $c_1$ ,  $c_2$  and  $c_3$  trade-off the misclassification of the training sample, the separation between classes and the complexity of the tree, respectively. These parameters should be carefully calibrated in order to construct *simple* decision trees with high predictive power, as can be seen in our computational experiments.

**4.3. Constraints.** The requirements on the relationships between the variables and the rationale of our model are described through the following constraints that define the mathematical programming model.

First of all, in order to adequately represent the maximum among the inverse margins of the splitting hyperplanes, we require:

$$\delta \geq \frac{1}{2} \|\omega_t\|_2^2, \forall t \in \tau_b. \quad (\text{C1})$$

Next, we impose how the splits are performed in the tree. To this end, we need to know which observations belong to a certain node  $t$  ( $z$ -variable) and how these observations are distributed with respect to the two fictitious classes to be separated ( $\alpha$ -variables). Gathering all these elements together, we use the following constraints to define the splits of the decision tree:

$$\omega'_t x_i + \omega_{t0} \geq 1 - e_{it} - (1 - h_{it}) \quad \forall i \in N, t \in \tau_b, \quad (\text{C2a})$$

$$\omega'_t x_i + \omega_{t0} \leq -1 + e_{it} + (1 - z_{it} + \alpha_{it}) \quad \forall i \in N, t \in \tau_b. \quad (\text{C2b})$$

According to this, constraint (C2a) is activated just in case the observation  $i$  belongs to the reference class and it is in node  $t$  ( $h_{it} = 1$ ). On the other hand, (C2b) is activated if  $i$  is allocated to node  $t$  ( $z_{it} = 1$ ) but it does not belong to the reference class ( $\alpha_{it} = 0$ ). Therefore, the reference class is located on the positive half space of hyperplane  $\mathcal{H}_t$ , while the other class is positioned in the negative half space, and at the same time, margin violations are regulated by the  $e_{it}$  variables.

To ensure the correct behaviour of the above constraints, we must correctly define the  $z_{it}$  variables. First, it is required that each observation belongs to exactly one node per level in the tree. This can be easily done by adding the usual assignment constraints to the problem at each of the levels,  $u \in U$ , of the tree:

$$\sum_{t \in u} z_{it} = 1 \quad \forall i \in N, u \in U. \quad (\text{C3})$$

Furthermore, we should enforce that if observation  $i$  is in node  $t$  ( $z_{it} = 1$ ), then observation  $i$  must also be in the parent node of  $t$ ,  $p(t)$  ( $z_{ip(t)} = 1$ ), and also observation  $i$  can not be in node  $t$  if it is not in its parent node ( $z_{ip(t)} = 0 \Rightarrow z_{it} = 0$ ). These implications can be obtained by means of the following constraints:

$$z_{it} \leq z_{ip(t)} \quad \forall i \in N, t = 2, \dots, T. \quad (\text{C4})$$

Nevertheless, the way observations descend through the tree needs a further analysis, since at this point they could just randomly define a path in the tree. Whenever an observation  $i$  is in the positive half space of the splitting hyperplane at node  $t$ ,  $\mathcal{H}_t$ , this observation should follow the right branch connecting to the child node of  $t$ . Otherwise, in case  $i$  is on the negative half space, it should follow the left branch. The knowledge on the side of the splitting hyperplane where an observation belongs to is encoded in the  $\alpha$ -variables. Then, in case  $i$  lies on the positive half space of  $\mathcal{H}_t$ ,  $\alpha_{it}$  will never be equal to zero since it would lead to a value of  $e_{it}$  greater than one, while  $e_{it} < 1$  is guaranteed in case  $\alpha_{it} = 1$ .

With the above observations, the constraints that assure the correct construction of the splitting hyperplanes with respect to the side of them where the observations belong to are the following:

$$z_{ip(t)} - z_{it} \leq \alpha_{ip(t)} \quad \forall i \in N, t \in \tau_{bl}, \quad (\text{C5a})$$

$$z_{ip(t)} - z_{it} \leq 1 - \alpha_{ip(t)} \quad \forall i \in N, t \in \tau_{br}. \quad (\text{C5b})$$

Constraints (C5a) assure that if observation  $i$  is on the parent node of an even node  $t$  ( $z_{ip(t)} = 1$ ), and  $i$  lies on the negative half space of  $\mathcal{H}_{p(t)}$  ( $\alpha_{ip(t)} = 0$ ), then  $z_{it}$  is enforced to be equal to one. As a result,  $\alpha_{ip(t)} = 0$  forces observation  $i$  to take the left branch in node  $t$ . Note that in case  $z_{ip(t)} = 1$ , and at the same time observation  $i$  is not in the left child node of  $t$  ( $z_{it} = 0$  for  $i \in \tau_{bl}$ ), then  $\alpha_{ip(t)} = 1$ , which means that observation  $i$  lies on the positive half space of  $\mathcal{H}_{p(t)}$ . Constraints (C5b) are analogous to (C5a) but allowing to adequately represent right branching nodes.

Moreover, two additional important elements need to be incorporated to complete our model: the tree complexity and the correct definition of misclassified observations. Note that in usual Optimal Classification Trees that do not use SVM-based splits, the complexity can be easily regulated by just imposing  $\|\omega_t\|_2^2 \leq Md_t$  (for a big enough  $M$  constant) in all the branch nodes, since in case a node is no further branched ( $d_t = 0$ ), the coefficients of the splitting hyperplane are set to zero. However, in our case, in which the splitting hyperplanes are SVM-based hyperplanes, these constraints are in conflict with constraints (C2a) and (C2b), since in case  $d_t = 0$  (and therefore  $\omega_t = 0$ ) it would not only imply that the coefficients  $\omega_t$  are equal to zero, but also that the distance based errors would be set to the maximum value of 1, i.e.,  $e_{it} = 1$  for every observation  $i$  in the node, even though these errors would not make any sense since observations would not be separated at the node. To overcome this issue, we consider the auxiliary binary variables  $h_{it} = z_{it}\alpha_{it}$  ( $h_{it}$  takes value 1 if observation  $i$  belongs to node  $t$  and lies in the positive half-space of the splitting hyperplane applied at node  $t$ ) and  $v_t$  (that takes value zero in case all the points in the node belong to the positive halfspace and one otherwise). The variables are adequately defined if the following constraints are incorporated to the model:

$$h_{it} \geq z_{it} + \alpha_{it} - 1, \quad \forall i \in N, t \in \tau_b, \quad (\text{C6a})$$

$$h_{it} \leq z_{it} - \alpha_{it} + 1, \quad \forall i \in N, t \in \tau_b, \quad (\text{C6b})$$

$$\sum_{i \in N} (z_{it} - h_{it}) \leq nv_t, \quad \forall t \in \tau_b, \quad (\text{C6c})$$

$$\sum_{i \in N} h_{it} \leq n(1 + d_t - v_t), \quad \forall t \in \tau_b, \quad (\text{C6d})$$

where constraints (C6a) and (C6b) are the linearization of the bilinear constraint  $h_{it} = z_{it}\alpha_{it}$ . On the other hand, Constraints (C6c) assure that in case  $v_t = 0$ , then all observations in node  $t$  belong to the positive halfspace of  $\mathcal{H}_t$ , and constraints (C6d) assure that if  $v_t = 1$  and the tree is pruned at node  $t$  ( $d_t = 0$ ), then those observations allocated to node  $t$  are placed in the negative halfspace defined by the splitting hyperplane. Thus, it implies that  $d_t$  takes value one if and only if the observations in node  $t$  are separated by  $\mathcal{H}_t$ , and therefore producing an effective split at the node.

Finally, in order to adequately represent the  $L_t$  variables (the ones that measure the number of misclassified observations at the leaf nodes) we use the constraints already incorporated in the OCT-H model in [8]. On the one hand, we assign each leaf node to a single class (the most popular class of the observations that belong to that node). We use the binary variable  $q_{kt}$  to check whether leaf node  $t \in \tau_l$  is assigned to class  $k = 1, \dots, K$ . The usual assignment constraints

are considered to assure that each node is assigned to exactly one class:

$$\sum_{k=1}^K q_{kt} = 1, \quad \forall t \in \tau_l. \quad (\text{C7})$$

The correct definition of the variable  $L_t$  is then guaranteed by the following set of constraints:

$$L_t \geq \sum_{i \in N} z_{it} - \sum_{i \in N} Y_{ik} z_{it} - n(1 - q_{kt}), \quad \forall k = 1, \dots, K, t \in \tau_l, \quad (\text{C8})$$

These constraints are activated if and only if  $q_{kt} = 1$ , i.e., if observations in node  $t$  are assigned to class  $k$ . In such a case, since  $L_t$  is being minimized in the objective function,  $L_t$  will be determined by the number of training observations in node  $t$  except those whose label is  $k$ , i.e., the number of missclassified observations in node  $t$  according to the  $k$ -class assignment.

Observe that the constant  $n$  in (C8) can be decreased and fixed to the maximum number of misclassified observations in the training sample. This number coincide with the difference between the number of observations in the training sample ( $n$ ) and the number of observations in the most represented class in the sample.

Summarizing the above paragraphs, the MOCTSVM can be formulated as the following MINLP problem:

$$\min \quad \delta + c_1 \sum_{t \in \tau_l} L_t + c_2 \sum_{i \in N} \sum_{t \in \tau} e_{it} + c_3 \sum_{t \in \tau} d_t \quad (\text{OBJ})$$

$$\text{s.t. } \delta \geq \frac{1}{2} \|\omega_t\|, \quad \forall t \in \tau_b, \quad (\text{C1})$$

$$\omega'_t x_i + \omega_{t0} \geq 1 - e_{it} - (2 - z_{it} - \alpha_{it}), \quad \forall i \in N, t \in \tau_b, \quad (\text{C2a})$$

$$\omega'_t x_i + \omega_{t0} \leq -1 + e_{it} + (1 - z_{it} + \alpha_{it}), \quad \forall i \in N, t \in \tau_b, \quad (\text{C2b})$$

$$\sum_{t \in u} z_{it} = 1, \quad \forall i \in N, u \in U, \quad (\text{C3})$$

$$z_{it} \leq z_{ip(t)}, \quad \forall i \in N, t = 2, \dots, T, \quad (\text{C4})$$

$$z_{ip(t)} - z_{it} \leq \alpha_{ip(t)}, \quad \forall i \in N, t \in \tau_{bl}, \quad (\text{C5a})$$

$$z_{ip(t)} - z_{it} \leq 1 - \alpha_{ip(t)}, \quad \forall i \in N, t \in \tau_{br}, \quad (\text{C5b})$$

$$h_{it} \geq z_{it} + \alpha_{it} - 1, \quad \forall i \in N, t \in \tau_b, \quad (\text{C6a})$$

$$h_{it} \leq z_{it} - \alpha_{it} + 1, \quad \forall i \in N, t \in \tau_b, \quad (\text{C6b})$$

$$\sum_{i \in N} (z_{it} - h_{it}) \leq nv_t, \quad \forall t \in \tau_b, \quad (\text{C6c})$$

$$\sum_{i \in N} h_{it} \leq n(1 + d_t - v_t), \quad \forall t \in \tau_b, \quad (\text{C6d})$$

$$\sum_{k=1}^K q_{kt} = 1, \quad \forall t \in \tau_l, \quad (\text{C7})$$

$$L_t \geq \sum_{i \in N} z_{it} - \sum_{i \in N} Y_{ik} z_{it} - n(1 - q_{kt}), \quad \forall k = 1, \dots, K, t \in \tau_l, \quad (\text{C8})$$

$$e_{it} \in \mathbb{R}^+, \alpha_{it}, h_{it} \in \{0, 1\}, \quad \forall i \in N, t \in \tau_b,$$

$$z_{it} \in \{0, 1\}, \quad \forall i \in N, t = 1, \dots, T,$$

$$q_{kt} \in \{0, 1\}, \quad \forall k = 1 \dots, K, t \in \tau_l,$$

$$\omega_t \in \mathbb{R}^p, \omega_{t0} \in \mathbb{R}, d_t \in \{0, 1\}, \quad \forall t = 1, \dots, T.$$

**4.4. Strengthening the model.** The MINLP formulation presented above is valid for our MOCTSVM model. However, it is a computationally costly problem, and although it can be solved by most of the off-the-shelf optimization solvers (as Gurobi, CPLEX or XPRESS), it is able to solve optimally only small to medium size instances. To improve its performance, the problem can be strengthened by means of valid inequalities which allows one to reduce the gap between the continuous relaxation of the problem and its optimal integer solution, being then able to solve larger instances in smaller CPU times. In what follows we describe some of these inequalities that we have incorporated to the MINLP formulation:

- If observations  $i$  and  $i'$  belongs to different nodes, they cannot be assigned to the same node for the remainder levels of the tree:

$$z_{is} + z_{i's} \leq z_{it} + z_{i't}, \forall t \in u, s \in u' u \leq u'$$

- If leaf nodes  $t$  and  $s$  are the result of proper splitting hyperplanes, then, both nodes cannot be assigned to the same class:

$$q_{kt} + q_{ks} \leq 2 - d_{p(t)}, \forall t, s = 2, \dots, T (t \neq s) \text{ with } p(t) = p(s), k = 1, \dots, K.$$

- Variable  $\alpha_{it}$  is enforced to take value 0 in case  $z_{it} = 0$ :

$$\alpha_{it} \leq z_{it}, \forall i \in N, t \in \tau_b.$$

- Variable  $h_{it}$  is not allowed to take value one if  $\alpha_{it}$  takes value zero:

$$h_{it} \leq \alpha_{it}, \forall i \in N, t \in \tau_b.$$

- There should be at least a leaf node to which each class is assigned to (assuming that each class is represented in the training sample). It also implies that the number of nodes to which a class is assigned is bounded as:

$$1 \leq \sum_{t \in \tau_l} q_{kt} \leq 2^D - 1, \forall k = 1, \dots, K.$$

In order to reduce the dimensionality and also to avoid symmetries of the MINLP problem, one can also apply some heuristic strategies to fix the values of some of the binary variables in a preprocessing phase. For instance, we choose  $i_0 \in \arg \max_{i \in N} |\{i' \in N : \|a_i - a_{i'}\| \leq \varepsilon \text{ and } y_i = y_{i'}\}|$ , that is, the observation with a maximum amount of observations in the same class *close enough* to it. Then, we fix to one all the variables  $z_{it_0}$  with  $i \in \{i' \in N : \|a_i - a_{i'}\| \leq \varepsilon \text{ and } y_i = y_{i'}\}$ , being  $t_0$  the first left leaf node of the tree (and fixing to zero the allocation of these points to the rest of the leaf nodes). Analogously, we fix also to one all the  $z$ -variables allocating observations to the last right leaf node of the tree for a subset of observations in the same class which are far enough from  $i_0$ , i.e.,  $z_{it_f} = 1$  for all  $i \in \{i' \in N : \|a_{i_f} - a_{i'}\| \leq \varepsilon \text{ and } y_{i_f} = y_{i'}\}$  where  $i_f = \arg \max_{i \in N} \|a_{i_0} - a_i\|$  (and fixing to zero the allocation of these points to the rest of the leaf nodes).

## 5. EXPERIMENTS

In order to analyze the performance of this new methodology we have run a series of experiments among different real datasets from UCI Machine learning Repository [3]. We have chosen twelve datasets with number of classes between two and seven. The dimension of these problems is reported in Table 3 by the tuple  $(n : \text{number of observations}, p : \text{number of features}, K : \text{number of classes})$ .

We have compared the MOCTSVM model with three other Classification Tree-based methodologies, namely CART, OCT and OCT-H. The maximum tree depth,  $D$ , for all the models was equal to 3, and the minimum number of observations per node in CART, OCT and OCT-H was equal to the 5% of the training size.

We have performed, for each instance a 5-fold cross validation scheme, i.e., datasets have been splitted into five random train-test partitions where one of the folds is used to build the model and the remaining are used to measure the accuracy of the predictions. Moreover, in order to avoid taking advantage of beneficial initial partitions, we have repeated the cross-validation scheme five times for all the datasets.

The CART method was coded in R using the `rpart` library. On the other hand, MOCTSVM, OCT and OCT-H were coded in Python and solved using the optimization solver Gurobi 8.1.1. All the experiments were run on a PC Intel Xeon E-2146G processor at 3.50GHz and 64GB of RAM. A time limit of 300 seconds was set for training the training folds. Although not all the problems were optimally solved within the time limit, as can be observed in Table 3, the results obtained with our model already outperform the other methods.

In order to calibrate the parameters of the different models that regulate the complexity of the tree, we have used different approaches. On the one hand,

for CART and OCT, since the maximum number of nodes for such a depth is  $2^D - 1 = 7$ , one can search for the tree with best complexity by searching in the grid  $\{1, \dots, 2^D - 1\}$  of possible active nodes. For OCT-H, we search the complexity regularization factor in the grid  $\{10^i : i = -5, \dots, 5\}$ . Finally, in MOCTSVM we used the same grid  $\{10^i : i = -5, \dots, 5\}$  for  $c_1$  and  $c_2$ , and  $\{10^i : i = -2, \dots, 2\}$  for  $c_3$ .

In Table 3 we report the results obtained in our experiments for all the models. The first column of the table indicates the identification of the dataset (together with its dimensionality). Second, for each of the methods that we have tested, we report the obtained average test accuracy and the standard deviation. We have highlighted in bold the best average test accuracies obtained for each dataset.

As can be observed, our method clearly outperforms in most of the instances the rest of the methods in terms of accuracy. Clearly, our model is designed to construct Optimal Classification Trees with larger separations between the classes, which results in better accuracies in the test sample. The datasets *Australian* and *BalanceScale* obtain their better results with OCT-H, but, as can be observed, the differences with respect the rest of the methods are tiny (it is the result of correctly classifying in the test sample just a few more observations than the rest of the methods). In that case, our method gets an accuracy almost as good as OCT-H. In the rest of the datasets, our method consistently gets better classifiers and for instance for *Dermatology* the difference with respect to the best classifiers among the others ranges in  $[4\%, 19\%]$ , for *Parkinson* the accuracy with our model is at least 6% better than the rest, for *Wine* we get 5% more accuracy than OCTH and 10% more than CART and for *Zoo* the accuracy of our model is more than 17% greater than the one obtained with CART.

Concerning the variability of our method, the standard deviations reported in Table 3 show that our results are, in average, more stable than the others, with small deviations with respect to the average accuracies. This behaviour differs from the one observed in CART or OCT, where larger deviations are obtained, implying that the accuracies highly depends of the test folder where the method is applied.

## 6. CONCLUSIONS AND FURTHER RESEARCH

We have presented in this paper a novel methodology to construct classifiers for multiclass instances by means of a Mathematical Programming model. The proposed method outputs a classification tree where the splits are based on SVM-based hyperplanes. At each branch node of the tree, a binary SVM hyperplane is constructed in which the observations are classified in two fictitious classes (the original classes are ignored in all the splitting nodes), but the global goodness of the tree is measured at the leaf nodes, where misclassification errors

|                           | CART             | OCT              | OCT-H                              | MOCTSVM                            | Diff             |
|---------------------------|------------------|------------------|------------------------------------|------------------------------------|------------------|
| Australian<br>(690,14,2)  | 85.54 $\pm$ 0.81 | 85.22 $\pm$ 1.27 | <b>85.65 <math>\pm</math> 1.02</b> | 85.27 $\pm$ 1.11                   | -0.38 $\pm$ 0.63 |
| BalanceScale<br>(625,4,3) | 69.55 $\pm$ 1.76 | 73.30 $\pm$ 1.20 | <b>90.43 <math>\pm</math> 1.07</b> | 89.53 $\pm$ 1.28                   | -0.90 $\pm$ 1.69 |
| Banknote<br>(1372,5,2)    | 89.27 $\pm$ 0.95 | 88.50 $\pm$ 1.17 | 98.89 $\pm$ 0.33                   | <b>98.91 <math>\pm</math> 0.46</b> | 0.02 $\pm$ 0.43  |
| BreastCancer<br>(683,9,2) | 92.69 $\pm$ 1.01 | 94.16 $\pm$ 0.54 | 95.10 $\pm$ 1.26                   | <b>96.27 <math>\pm</math> 0.64</b> | 1.17 $\pm$ 1.39  |
| Dermatology<br>(358,34,6) | 75.69 $\pm$ 3.60 | 77.82 $\pm$ 4.34 | 91.41 $\pm$ 2.83                   | <b>95.39 <math>\pm</math> 1.47</b> | 3.98 $\pm$ 2.74  |
| Heart<br>(294,13,5)       | 64.37 $\pm$ 1.48 | 65.14 $\pm$ 1.57 | 64.30 $\pm$ 1.79                   | <b>66.41 <math>\pm</math> 1.54</b> | 1.26 $\pm$ 1.38  |
| Iris<br>(150,4,3)         | 94.26 $\pm$ 1.90 | 95.37 $\pm$ 0.97 | 95.64 $\pm$ 1.46                   | <b>95.72 <math>\pm</math> 1.79</b> | 0.08 $\pm$ 1.70  |
| Parkinson<br>(240,40,2)   | 72.29 $\pm$ 4.05 | 73.53 $\pm$ 2.26 | 74.92 $\pm$ 3.01                   | <b>80.83 <math>\pm</math> 1.89</b> | 5.91 $\pm$ 3.06  |
| Seeds<br>(210,7,3)        | 86.36 $\pm$ 4.02 | 88.52 $\pm$ 2.69 | 91.12 $\pm$ 2.99                   | <b>92.98 <math>\pm</math> 1.82</b> | 1.85 $\pm$ 2.23  |
| Teaching<br>(150,5,3)     | 41.91 $\pm$ 5.64 | 48.35 $\pm$ 3.80 | 48.09 $\pm$ 2.92                   | <b>48.62 <math>\pm</math> 3.37</b> | 0.26 $\pm$ 4.60  |
| Thyroid<br>(215,5,3)      | 89.77 $\pm$ 2.37 | 92.43 $\pm$ 2.12 | 92.46 $\pm$ 2.49                   | <b>94.57 <math>\pm</math> 2.08</b> | 2.11 $\pm$ 3.10  |
| Wine<br>(178,13,3)        | 84.52 $\pm$ 2.66 | 92.22 $\pm$ 3.41 | 89.35 $\pm$ 3.71                   | <b>94.13 <math>\pm</math> 1.78</b> | 1.90 $\pm$ 3.37  |
| Zoo<br>(101,16,7)         | 74.96 $\pm$ 5.79 | 87.75 $\pm$ 1.99 | 89.11 $\pm$ 2.58                   | <b>92.31 <math>\pm</math> 2.15</b> | 3.20 $\pm$ 2.95  |

TABLE 3. Average accuracies ( $\pm$  standard deviations) obtained in our computational experiments.

are minimized. Also, the model minimizes the complexity of the tree together with the two elements that appear in SVM-approaches: margin separation and distance-based misclassifying errors. We have run an extensive battery of computational experiments that shows that our method outperforms most of the Decision Tree-based methodologies both in accuracy and stability.

Future research lines on this topic include the analysis of nonlinear splits when branching in MOCTSVM, both using kernel tools derived from SVM classifiers or specific families of nonlinear separators. This approach will result into more flexible classifiers able to capture the nonlinear trends of many real-life datasets. Additionally, we also plan to incorporate features selection in our method in order to construct highly predictive but also more interpretable classification tools.

## ACKNOWLEDGEMENTS

This research has been partially supported by Spanish Ministerio de Ciencia e Innovación, Agencia Estatal de Investigación/FEDER grant number PID2020-114594GBC21, Junta de Andalucía projects P18-FR-1422 and projects FEDERUS-1256951, BFQM-322-UGR20, CEI-3-FQM331 and NetmeetData-Ayudas Fundación BBVA a equipos de investigación científica 2019. The first author was also partially supported by the IMAG-Maria de Maeztu grant CEX2020-001105-M /AEI /10.13039/501100011033.

## REFERENCES

- [1] AGARWAL, N., BALASUBRAMANIAN, V. N., AND JAWAHAR, C. Improving multiclass classification by deep networks using dagsvm and triplet loss. *Pattern Recognition Letters* 112 (2018), 184–190.
- [2] ALLWEIN, E. L., SCHAPIRE, R. E., AND SINGER, Y. Reducing multiclass to binary: A unifying approach for margin classifiers. *Journal of machine learning research* 1, Dec (2000), 113–141.
- [3] ASUNCION, A., AND NEWMAN, D. Uci machine learning repository, 2007.
- [4] BAHLMANN, C., HAASDONK, B., AND BURKHARDT, H. Online handwriting recognition with support vector machines-a kernel approach. In *Proceedings Eighth International Workshop on Frontiers in Handwriting Recognition* (2002), IEEE, pp. 49–54.
- [5] BALDOMERO-NARANJO, M., MARTÍNEZ-MERINO, L. I., AND RODRÍGUEZ-CHÍA, A. M. Tightening big ms in integer programming formulations for support vector machines with ramp loss. *European Journal of Operational Research* 286, 1 (2020), 84–100.
- [6] BALDOMERO-NARANJO, M., MARTÍNEZ-MERINO, L. I., AND RODRÍGUEZ-CHÍA, A. M. A robust svm-based approach with feature selection and outliers detection for classification problems. *Expert Systems with Applications* 178 (2021), 115017.
- [7] BENÍTEZ-PEÑA, S., BLANQUERO, R., CARRIZOSA, E., AND RAMÍREZ-COBO, P. Cost-sensitive feature selection for support vector machines. *Computers & Operations Research* 106 (2019), 169–178.
- [8] BERTSIMAS, D., AND DUNN, J. Optimal classification trees. *Machine Learning* 106, 7 (2017), 1039–1082.
- [9] BERTSIMAS, D., AND DUNN, J. W. Machine learning under a modern optimization lens, 2019.
- [10] BLANCO, V., JAPÓN, A., AND PUERTO, J. A mathematical programming approach to binary supervised classification with label noise. *arXiv preprint arXiv:2004.10170* (2020).
- [11] BLANCO, V., JAPÓN, A., AND PUERTO, J. Optimal arrangements of hyperplanes for svm-based multiclass classification. *Advances in Data Analysis and Classification* 14, 1 (2020), 175–199.
- [12] BLANCO, V., JAPÓN, A., AND PUERTO, J. Robust optimal classification trees under noisy labels. *Advances in Data Analysis and Classification* (2021).
- [13] BLANCO, V., PUERTO, J., AND RODRIGUEZ-CHIA, A. M. On lp-support vector machines and multidimensional kernels. *J. Mach. Learn. Res.* 21 (2020), 14–1.
- [14] BLANQUERO, R., CARRIZOSA, E., MOLERO-RÍO, C., AND MORALES, D. R. Optimal randomized classification trees. *Computers & Operations Research* 132 (2021), 105281.

- [15] BREIMAN, L., FRIEDMAN, J., OLSHEN, R., AND STONE, C. Classification and regression trees, 1984.
- [16] CARRIZOSA, E., MOLERO-RÍO, C., AND MORALES, D. R. Mathematical optimization in classification and regression trees. *Top* 29, 1 (2021), 5–33.
- [17] CORTES, C., AND VAPNIK, V. Support-vector networks. *Machine learning* 20, 3 (1995), 273–297.
- [18] COVER, T., AND HART, P. Nearest neighbor pattern classification. *IEEE transactions on information theory* 13, 1 (1967), 21–27.
- [19] CRAMMER, K., AND SINGER, Y. On the algorithmic implementation of multiclass kernel-based vector machines. *Journal of machine learning research* 2, Dec (2001), 265–292.
- [20] DIETTERICH, T. G., AND BAKIRI, G. Solving multiclass learning problems via error-correcting output codes. *Journal of artificial intelligence research* 2 (1994), 263–286.
- [21] DU, M., LIU, N., AND HU, X. Techniques for interpretable machine learning. *Communications of the ACM* 63, 1 (2019), 68–77.
- [22] EITRICH, T., AND LANG, B. Efficient optimization of support vector machine learning parameters for unbalanced datasets. *Journal of computational and applied mathematics* 196, 2 (2006), 425–436.
- [23] FRIEDMAN, J., HASTIE, T., AND TIBSHIRANI, R. The elements of statistical learning, 2001.
- [24] GAN, J., LI, J., AND XIE, Y. Robust svm for cost-sensitive learning. *Neural Processing Letters* (2021), 1–22.
- [25] GAUDIOSO, M., GORGONE, E., LABBÉ, M., AND RODRÍGUEZ-CHÍA, A. M. Lagrangian relaxation for svm feature selection. *Computers & Operations Research* 87 (2017), 137–145.
- [26] GUERMEUR, Y., AND MONFRINI, E. A quadratic loss multi-class svm for which a radius-margin bound applies. *Informatica* 22, 1 (2011), 73–96.
- [27] GÜNLÜK, O., KALAGNANAM, J., MENICKELLY, M., AND SCHEINBERG, K. Optimal decision trees for categorical data via integer programming. *arXiv preprint arXiv:1612.03225* (2018).
- [28] HARRIS, T. Quantitative credit risk assessment using support vector machines: Broad versus narrow default definitions. *Expert Systems with Applications* 40, 11 (2013), 4404–4413.
- [29] HASTIE, T. J., AND TIBSHIRANI, R. J. Generalized additive models, 2017.
- [30] JIMÉNEZ-CORDERO, A., MORALES, J. M., AND PINEDA, S. A novel embedded min-max approach for feature selection in nonlinear support vector machine classification. *European Journal of Operational Research* 293, 1 (2021), 24–35.
- [31] KAŠĆELAN, V., KAŠĆELAN, L., AND NOVVIĆ BURIĆ, M. A nonparametric data mining approach for risk prediction in car insurance: a case study from the montenegrin market. *Economic research-Ekonomska istraživanja* 29, 1 (2016), 545–558.
- [32] LAUER, F., AND GUERMEUR, Y. Msvmpack: a multi-class support vector machine package. *The Journal of Machine Learning Research* 12 (2011), 2293–2296.
- [33] LEE, Y., LIN, Y., AND WAHBA, G. Multicategory support vector machines: Theory and application to the classification of microarray data and satellite radiance data. *Journal of the American Statistical Association* 99, 465 (2004), 67–81.
- [34] LETHAM, B., RUDIN, C., MCCORMICK, T. H., AND MADIGAN, D. Interpretable classifiers using rules and bayesian analysis: Building a better stroke prediction model. *The Annals of Applied Statistics* 9, 3 (2015), 1350–1371.

- [35] LEWIS, D. D. Naive (bayes) at forty: The independence assumption in information retrieval. In *European conference on machine learning* (1998), Springer, pp. 4–15.
- [36] MAJID, A., ALI, S., IQBAL, M., AND KAUSAR, N. Prediction of human breast and colon cancers from imbalanced data using nearest neighbor and support vector machines. *Computer methods and programs in biomedicine* 113, 3 (2014), 792–808.
- [37] MEYER, D., DIMITRIADOU, E., HORNIK, K., WEINGESSEL, A., LEISCH, F., CHANG, C., AND LIN, C. Misc functions of the department of statistics, probability theory group (formerly: E1071). *Package e1071. TU Wien* (2015).
- [38] PEDREGOSA, F., VAROQUAUX, G., GRAMFORT, A., MICHEL, V., THIRION, B., GRISEL, O., BLONDEL, M., PRETTENHOFER, P., WEISS, R., DUBOURG, V., ET AL. Scikit-learn: Machine learning in python. *the Journal of machine Learning research* 12 (2011), 2825–2830.
- [39] QUINLAN, J. Machine learning and id3. *Los Altos: Morgan Kauffman* (1996).
- [40] QUINLAN, R. C4. 5. *Programs for machine learning* (1993).
- [41] RADHIMEENAKSHI, S. Classification and prediction of heart disease risk using data mining techniques of support vector machine and artificial neural network. In *2016 3rd International Conference on Computing for Sustainable Global Development (INDIACom)* (2016), IEEE, pp. 3107–3111.
- [42] TANG, X., AND XU, A. Multi-class classification using kernel density estimation on k-nearest neighbours. *Electronics Letters* 52, 8 (2016), 600–602.
- [43] VAN DEN BURG, G., AND GROENEN, P. Gensvm: A generalized multiclass support vector machine. *Journal of Machine Learning Research* 17 (2016), 1–42.
- [44] WESTON, J., AND WATKINS, C. Support vector machines for multi-class pattern recognition. In *Esann* (1999), vol. 99, pp. 219–224.



# 9

## Support Vector Machines

In this chapter, we discuss the *support vector machine* (SVM), an approach for classification that was developed in the computer science community in the 1990s and that has grown in popularity since then. SVMs have been shown to perform well in a variety of settings, and are often considered one of the best “out of the box” classifiers.

The support vector machine is a generalization of a simple and intuitive classifier called the *maximal margin classifier*, which we introduce in Section 9.1. Though it is elegant and simple, we will see that this classifier unfortunately cannot be applied to most data sets, since it requires that the classes be separable by a linear boundary. In Section 9.2, we introduce the *support vector classifier*, an extension of the maximal margin classifier that can be applied in a broader range of cases. Section 9.3 introduces the *support vector machine*, which is a further extension of the support vector classifier in order to accommodate non-linear class boundaries. Support vector machines are intended for the binary classification setting in which there are two classes; in Section 9.4 we discuss extensions of support vector machines to the case of more than two classes. In Section 9.5 we discuss the close connections between support vector machines and other statistical methods such as logistic regression.

People often loosely refer to the maximal margin classifier, the support vector classifier, and the support vector machine as “support vector machines”. To avoid confusion, we will carefully distinguish between these three notions in this chapter.

## 9.1 Maximal Margin Classifier

In this section, we define a hyperplane and introduce the concept of an optimal separating hyperplane.

### 9.1.1 What Is a Hyperplane?

In a  $p$ -dimensional space, a *hyperplane* is a flat affine subspace of dimension  $p - 1$ .<sup>1</sup> For instance, in two dimensions, a hyperplane is a flat one-dimensional subspace—in other words, a line. In three dimensions, a hyperplane is a flat two-dimensional subspace—that is, a plane. In  $p > 3$  dimensions, it can be hard to visualize a hyperplane, but the notion of a  $(p - 1)$ -dimensional flat subspace still applies.

The mathematical definition of a hyperplane is quite simple. In two dimensions, a hyperplane is defined by the equation

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0 \quad (9.1)$$

for parameters  $\beta_0, \beta_1$ , and  $\beta_2$ . When we say that (9.1) “defines” the hyperplane, we mean that any  $X = (X_1, X_2)^T$  for which (9.1) holds is a point on the hyperplane. Note that (9.1) is simply the equation of a line, since indeed in two dimensions a hyperplane is a line.

Equation 9.1 can be easily extended to the  $p$ -dimensional setting:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p = 0 \quad (9.2)$$

defines a  $p$ -dimensional hyperplane, again in the sense that if a point  $X = (X_1, X_2, \dots, X_p)^T$  in  $p$ -dimensional space (i.e. a vector of length  $p$ ) satisfies (9.2), then  $X$  lies on the hyperplane.

Now, suppose that  $X$  does not satisfy (9.2); rather,

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p > 0. \quad (9.3)$$

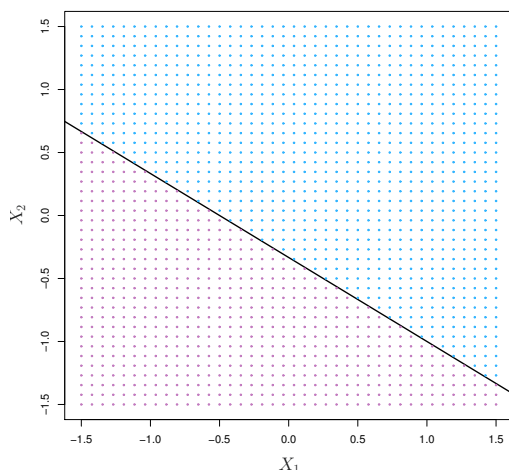
Then this tells us that  $X$  lies to one side of the hyperplane. On the other hand, if

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p < 0, \quad (9.4)$$

then  $X$  lies on the other side of the hyperplane. So we can think of the hyperplane as dividing  $p$ -dimensional space into two halves. One can easily determine on which side of the hyperplane a point lies by simply calculating the sign of the left hand side of (9.2). A hyperplane in two-dimensional space is shown in Figure 9.1.

---

<sup>1</sup>The word *affine* indicates that the subspace need not pass through the origin.



**FIGURE 9.1.** The hyperplane  $1 + 2X_1 + 3X_2 = 0$  is shown. The blue region is the set of points for which  $1 + 2X_1 + 3X_2 > 0$ , and the purple region is the set of points for which  $1 + 2X_1 + 3X_2 < 0$ .

### 9.1.2 Classification Using a Separating Hyperplane

Now suppose that we have a  $n \times p$  data matrix  $\mathbf{X}$  that consists of  $n$  training observations in  $p$ -dimensional space,

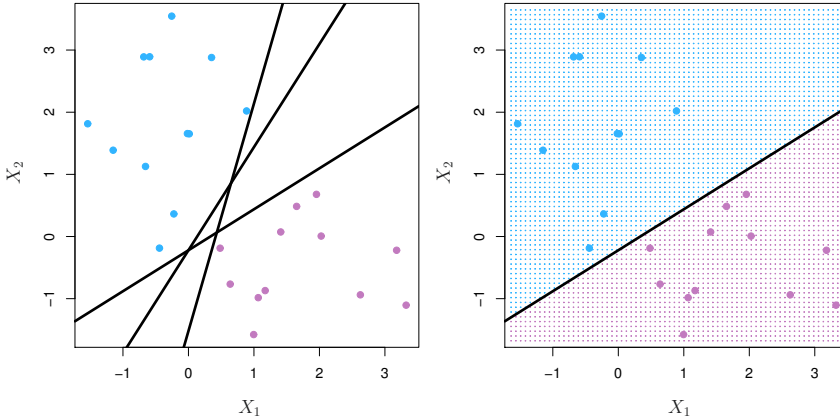
$$x_1 = \begin{pmatrix} x_{11} \\ \vdots \\ x_{1p} \end{pmatrix}, \dots, x_n = \begin{pmatrix} x_{n1} \\ \vdots \\ x_{np} \end{pmatrix}, \quad (9.5)$$

and that these observations fall into two classes—that is,  $y_1, \dots, y_n \in \{-1, 1\}$  where  $-1$  represents one class and  $1$  the other class. We also have a test observation, a  $p$ -vector of observed features  $x^* = (x_1^* \dots x_p^*)^T$ . Our goal is to develop a classifier based on the training data that will correctly classify the test observation using its feature measurements. We have seen a number of approaches for this task, such as linear discriminant analysis and logistic regression in Chapter 4, and classification trees, bagging, and boosting in Chapter 8. We will now see a new approach that is based upon the concept of a *separating hyperplane*.

Suppose that it is possible to construct a hyperplane that separates the training observations perfectly according to their class labels. Examples of three such *separating hyperplanes* are shown in the left-hand panel of Figure 9.2. We can label the observations from the blue class as  $y_i = 1$  and those from the purple class as  $y_i = -1$ . Then a separating hyperplane has the property that

separating  
hyperplane

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} > 0 \text{ if } y_i = 1, \quad (9.6)$$



**FIGURE 9.2.** Left: There are two classes of observations, shown in blue and in purple, each of which has measurements on two variables. Three separating hyperplanes, out of many possible, are shown in black. Right: A separating hyperplane is shown in black. The blue and purple grid indicates the decision rule made by a classifier based on this separating hyperplane: a test observation that falls in the blue portion of the grid will be assigned to the blue class, and a test observation that falls into the purple portion of the grid will be assigned to the purple class.

and

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} < 0 \text{ if } y_i = -1. \quad (9.7)$$

Equivalently, a separating hyperplane has the property that

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) > 0 \quad (9.8)$$

for all  $i = 1, \dots, n$ .

If a separating hyperplane exists, we can use it to construct a very natural classifier: a test observation is assigned a class depending on which side of the hyperplane it is located. The right-hand panel of Figure 9.2 shows an example of such a classifier. That is, we classify the test observation  $x^*$  based on the sign of  $f(x^*) = \beta_0 + \beta_1 x_1^* + \beta_2 x_2^* + \cdots + \beta_p x_p^*$ . If  $f(x^*)$  is positive, then we assign the test observation to class 1, and if  $f(x^*)$  is negative, then we assign it to class  $-1$ . We can also make use of the *magnitude* of  $f(x^*)$ . If  $f(x^*)$  is far from zero, then this means that  $x^*$  lies far from the hyperplane, and so we can be confident about our class assignment for  $x^*$ . On the other hand, if  $f(x^*)$  is close to zero, then  $x^*$  is located near the hyperplane, and so we are less certain about the class assignment for  $x^*$ . Not surprisingly, and as we see in Figure 9.2, a classifier that is based on a separating hyperplane leads to a linear decision boundary.

### 9.1.3 The Maximal Margin Classifier

In general, if our data can be perfectly separated using a hyperplane, then there will in fact exist an infinite number of such hyperplanes. This is because a given separating hyperplane can usually be shifted a tiny bit up or down, or rotated, without coming into contact with any of the observations. Three possible separating hyperplanes are shown in the left-hand panel of Figure 9.2. In order to construct a classifier based upon a separating hyperplane, we must have a reasonable way to decide which of the infinite possible separating hyperplanes to use.

A natural choice is the *maximal margin hyperplane* (also known as the *optimal separating hyperplane*), which is the separating hyperplane that is farthest from the training observations. That is, we can compute the (perpendicular) distance from each training observation to a given separating hyperplane; the smallest such distance is the minimal distance from the observations to the hyperplane, and is known as the *margin*. The maximal margin hyperplane is the separating hyperplane for which the margin is largest—that is, it is the hyperplane that has the farthest minimum distance to the training observations. We can then classify a test observation based on which side of the maximal margin hyperplane it lies. This is known as the *maximal margin classifier*. We hope that a classifier that has a large margin on the training data will also have a large margin on the test data, and hence will classify the test observations correctly. Although the maximal margin classifier is often successful, it can also lead to overfitting when  $p$  is large.

maximal  
margin  
hyperplane  
optimal  
separating  
hyperplane  
margin

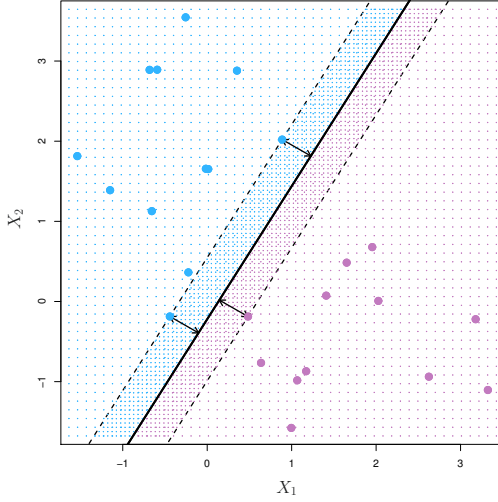
maximal  
margin  
classifier

If  $\beta_0, \beta_1, \dots, \beta_p$  are the coefficients of the maximal margin hyperplane, then the maximal margin classifier classifies the test observation  $x^*$  based on the sign of  $f(x^*) = \beta_0 + \beta_1 x_1^* + \beta_2 x_2^* + \dots + \beta_p x_p^*$ .

Figure 9.3 shows the maximal margin hyperplane on the data set of Figure 9.2. Comparing the right-hand panel of Figure 9.2 to Figure 9.3, we see that the maximal margin hyperplane shown in Figure 9.3 does indeed result in a greater minimal distance between the observations and the separating hyperplane—that is, a larger margin. In a sense, the maximal margin hyperplane represents the mid-line of the widest “slab” that we can insert between the two classes.

Examining Figure 9.3, we see that three training observations are equidistant from the maximal margin hyperplane and lie along the dashed lines indicating the width of the margin. These three observations are known as *support vectors*, since they are vectors in  $p$ -dimensional space (in Figure 9.3,  $p = 2$ ) and they “support” the maximal margin hyperplane in the sense that if these points were moved slightly then the maximal margin hyperplane would move as well. Interestingly, the maximal margin hyperplane depends directly on the support vectors, but not on the other observations: a movement to any of the other observations would not affect the separating hyperplane, provided that the observation’s movement does not cause it to

support  
vector



**FIGURE 9.3.** There are two classes of observations, shown in blue and in purple. The maximal margin hyperplane is shown as a solid line. The margin is the distance from the solid line to either of the dashed lines. The two blue points and the purple point that lie on the dashed lines are the support vectors, and the distance from those points to the hyperplane is indicated by arrows. The purple and blue grid indicates the decision rule made by a classifier based on this separating hyperplane.

cross the boundary set by the margin. The fact that the maximal margin hyperplane depends directly on only a small subset of the observations is an important property that will arise later in this chapter when we discuss the support vector classifier and support vector machines.

#### 9.1.4 Construction of the Maximal Margin Classifier

We now consider the task of constructing the maximal margin hyperplane based on a set of  $n$  training observations  $x_1, \dots, x_n \in \mathbb{R}^p$  and associated class labels  $y_1, \dots, y_n \in \{-1, 1\}$ . Briefly, the maximal margin hyperplane is the solution to the optimization problem

$$\underset{\beta_0, \beta_1, \dots, \beta_p, M}{\text{maximize}} \quad M \quad (9.9)$$

$$\text{subject to} \quad \sum_{j=1}^p \beta_j^2 = 1, \quad (9.10)$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M \quad \forall i = 1, \dots, n. \quad (9.11)$$

This optimization problem (9.9)–(9.11) is actually simpler than it looks. First of all, the constraint in (9.11) that

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M \quad \forall i = 1, \dots, n$$

guarantees that each observation will be on the correct side of the hyperplane, provided that  $M$  is positive. (Actually, for each observation to be on the correct side of the hyperplane we would simply need  $y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) > 0$ , so the constraint in (9.11) in fact requires that each observation be on the correct side of the hyperplane, with some cushion, provided that  $M$  is positive.)

Second, note that (9.10) is not really a constraint on the hyperplane, since if  $\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} = 0$  defines a hyperplane, then so does  $k(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) = 0$  for any  $k \neq 0$ . However, (9.10) adds meaning to (9.11); one can show that with this constraint the perpendicular distance from the  $i$ th observation to the hyperplane is given by

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}).$$

Therefore, the constraints (9.10) and (9.11) ensure that each observation is on the correct side of the hyperplane and at least a distance  $M$  from the hyperplane. Hence,  $M$  represents the margin of our hyperplane, and the optimization problem chooses  $\beta_0, \beta_1, \dots, \beta_p$  to maximize  $M$ . This is exactly the definition of the maximal margin hyperplane! The problem (9.9)–(9.11) can be solved efficiently, but details of this optimization are outside of the scope of this book.

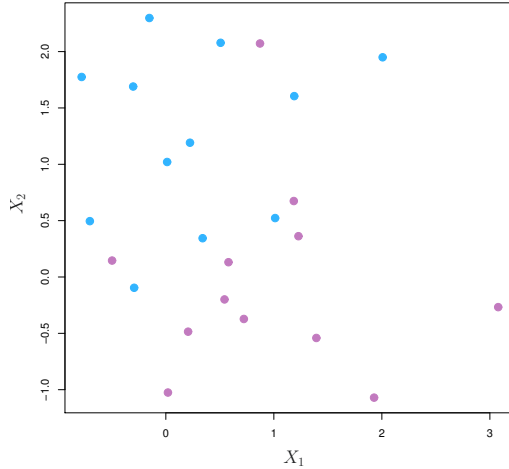
### 9.1.5 The Non-separable Case

The maximal margin classifier is a very natural way to perform classification, *if a separating hyperplane exists*. However, as we have hinted, in many cases no separating hyperplane exists, and so there is no maximal margin classifier. In this case, the optimization problem (9.9)–(9.11) has no solution with  $M > 0$ . An example is shown in Figure 9.4. In this case, we cannot *exactly* separate the two classes. However, as we will see in the next section, we can extend the concept of a separating hyperplane in order to develop a hyperplane that *almost* separates the classes, using a so-called *soft margin*. The generalization of the maximal margin classifier to the non-separable case is known as the *support vector classifier*.

## 9.2 Support Vector Classifiers

### 9.2.1 Overview of the Support Vector Classifier

In Figure 9.4, we see that observations that belong to two classes are not necessarily separable by a hyperplane. In fact, even if a separating hyperplane does exist, then there are instances in which a classifier based on a separating hyperplane might not be desirable. A classifier based on a separating hyperplane will necessarily perfectly classify all of the training



**FIGURE 9.4.** *There are two classes of observations, shown in blue and in purple. In this case, the two classes are not separable by a hyperplane, and so the maximal margin classifier cannot be used.*

observations; this can lead to sensitivity to individual observations. An example is shown in Figure 9.5. The addition of a single observation in the right-hand panel of Figure 9.5 leads to a dramatic change in the maximal margin hyperplane. The resulting maximal margin hyperplane is not satisfactory—for one thing, it has only a tiny margin. This is problematic because as discussed previously, the distance of an observation from the hyperplane can be seen as a measure of our confidence that the observation was correctly classified. Moreover, the fact that the maximal margin hyperplane is extremely sensitive to a change in a single observation suggests that it may have overfit the training data.

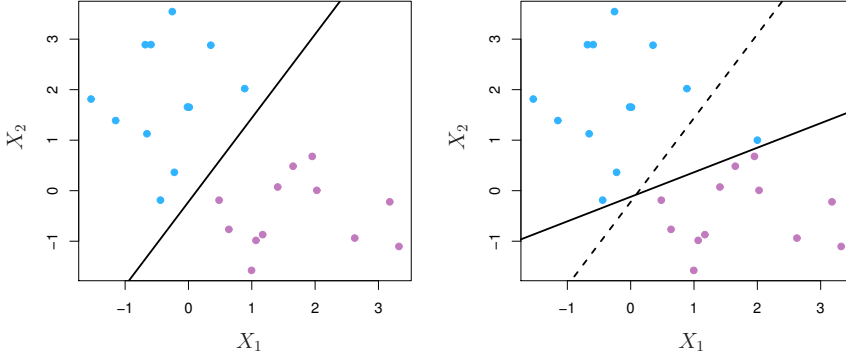
In this case, we might be willing to consider a classifier based on a hyperplane that does *not* perfectly separate the two classes, in the interest of

- Greater robustness to individual observations, and
- Better classification of *most* of the training observations.

That is, it could be worthwhile to misclassify a few training observations in order to do a better job in classifying the remaining observations.

The *support vector classifier*, sometimes called a *soft margin classifier*, does exactly this. Rather than seeking the largest possible margin so that every observation is not only on the correct side of the hyperplane but also on the correct side of the margin, we instead allow some observations to be on the incorrect side of the margin, or even the incorrect side of the hyperplane. (The margin is *soft* because it can be violated by some of the training observations.) An example is shown in the left-hand panel

support  
vector  
classifier  
soft margin  
classifier



**FIGURE 9.5.** Left: Two classes of observations are shown in blue and in purple, along with the maximal margin hyperplane. Right: An additional blue observation has been added, leading to a dramatic shift in the maximal margin hyperplane shown as a solid line. The dashed line indicates the maximal margin hyperplane that was obtained in the absence of this additional point.

of Figure 9.6. Most of the observations are on the correct side of the margin. However, a small subset of the observations are on the wrong side of the margin.

An observation can be not only on the wrong side of the margin, but also on the wrong side of the hyperplane. In fact, when there is no separating hyperplane, such a situation is inevitable. Observations on the wrong side of the hyperplane correspond to training observations that are misclassified by the support vector classifier. The right-hand panel of Figure 9.6 illustrates such a scenario.

### 9.2.2 Details of the Support Vector Classifier

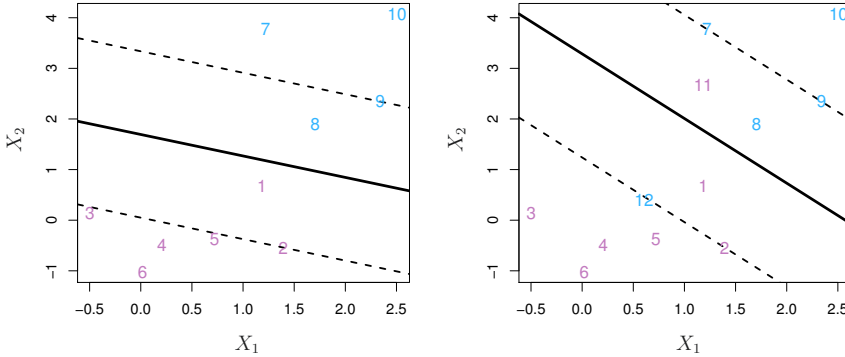
The support vector classifier classifies a test observation depending on which side of a hyperplane it lies. The hyperplane is chosen to correctly separate most of the training observations into the two classes, but may misclassify a few observations. It is the solution to the optimization problem

$$\underset{\beta_0, \beta_1, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n, M}{\text{maximize}} \quad M \quad (9.12)$$

$$\text{subject to} \quad \sum_{j=1}^p \beta_j^2 = 1, \quad (9.13)$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M(1 - \epsilon_i), \quad (9.14)$$

$$\epsilon_i \geq 0, \quad \sum_{i=1}^n \epsilon_i \leq C, \quad (9.15)$$



**FIGURE 9.6.** Left: A support vector classifier was fit to a small data set. The hyperplane is shown as a solid line and the margins are shown as dashed lines. Purple observations: Observations 3, 4, 5, and 6 are on the correct side of the margin, observation 2 is on the margin, and observation 1 is on the wrong side of the margin. Blue observations: Observations 7 and 10 are on the correct side of the margin, observation 9 is on the margin, and observation 8 is on the wrong side of the margin. No observations are on the wrong side of the hyperplane. Right: Same as left panel with two additional points, 11 and 12. These two observations are on the wrong side of the hyperplane and the wrong side of the margin.

where  $C$  is a nonnegative tuning parameter. As in (9.11),  $M$  is the width of the margin; we seek to make this quantity as large as possible. In (9.14),  $\epsilon_1, \dots, \epsilon_n$  are *slack variables* that allow individual observations to be on the wrong side of the margin or the hyperplane; we will explain them in greater detail momentarily. Once we have solved (9.12)–(9.15), we classify a test observation  $x^*$  as before, by simply determining on which side of the hyperplane it lies. That is, we classify the test observation based on the sign of  $f(x^*) = \beta_0 + \beta_1 x_1^* + \dots + \beta_p x_p^*$ .

slack  
variable

The problem (9.12)–(9.15) seems complex, but insight into its behavior can be made through a series of simple observations presented below. First of all, the slack variable  $\epsilon_i$  tells us where the  $i$ th observation is located, relative to the hyperplane and relative to the margin. If  $\epsilon_i = 0$  then the  $i$ th observation is on the correct side of the margin, as we saw in Section 9.1.4. If  $\epsilon_i > 0$  then the  $i$ th observation is on the wrong side of the margin, and we say that the  $i$ th observation has *violated* the margin. If  $\epsilon_i > 1$  then it is on the wrong side of the hyperplane.

We now consider the role of the tuning parameter  $C$ . In (9.15),  $C$  bounds the sum of the  $\epsilon_i$ 's, and so it determines the number and severity of the violations to the margin (and to the hyperplane) that we will tolerate. We can think of  $C$  as a *budget* for the amount that the margin can be violated by the  $n$  observations. If  $C = 0$  then there is no budget for violations to the margin, and it must be the case that  $\epsilon_1 = \dots = \epsilon_n = 0$ , in which case (9.12)–(9.15) simply amounts to the maximal margin hyperplane optimiza-

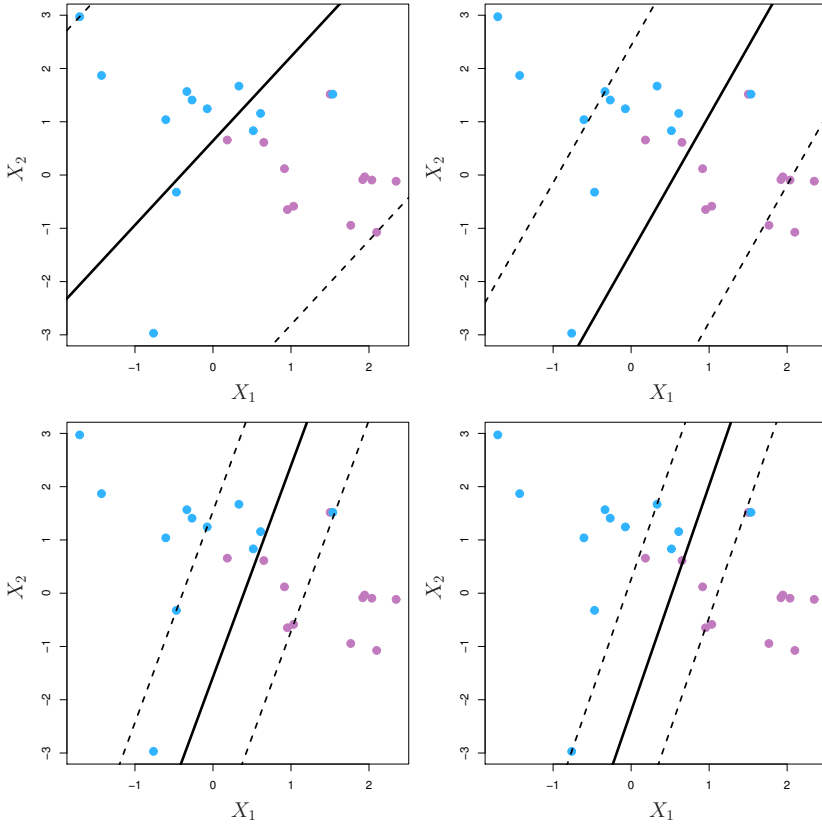
tion problem (9.9)–(9.11). (Of course, a maximal margin hyperplane exists only if the two classes are separable.) For  $C > 0$  no more than  $C$  observations can be on the wrong side of the hyperplane, because if an observation is on the wrong side of the hyperplane then  $\epsilon_i > 1$ , and (9.15) requires that  $\sum_{i=1}^n \epsilon_i \leq C$ . As the budget  $C$  increases, we become more tolerant of violations to the margin, and so the margin will widen. Conversely, as  $C$  decreases, we become less tolerant of violations to the margin and so the margin narrows. An example is shown in Figure 9.7.

In practice,  $C$  is treated as a tuning parameter that is generally chosen via cross-validation. As with the tuning parameters that we have seen throughout this book,  $C$  controls the bias-variance trade-off of the statistical learning technique. When  $C$  is small, we seek narrow margins that are rarely violated; this amounts to a classifier that is highly fit to the data, which may have low bias but high variance. On the other hand, when  $C$  is larger, the margin is wider and we allow more violations to it; this amounts to fitting the data less hard and obtaining a classifier that is potentially more biased but may have lower variance.

The optimization problem (9.12)–(9.15) has a very interesting property: it turns out that only observations that either lie on the margin or that violate the margin will affect the hyperplane, and hence the classifier obtained. In other words, an observation that lies strictly on the correct side of the margin does not affect the support vector classifier! Changing the position of that observation would not change the classifier at all, provided that its position remains on the correct side of the margin. Observations that lie directly on the margin, or on the wrong side of the margin for their class, are known as *support vectors*. These observations do affect the support vector classifier.

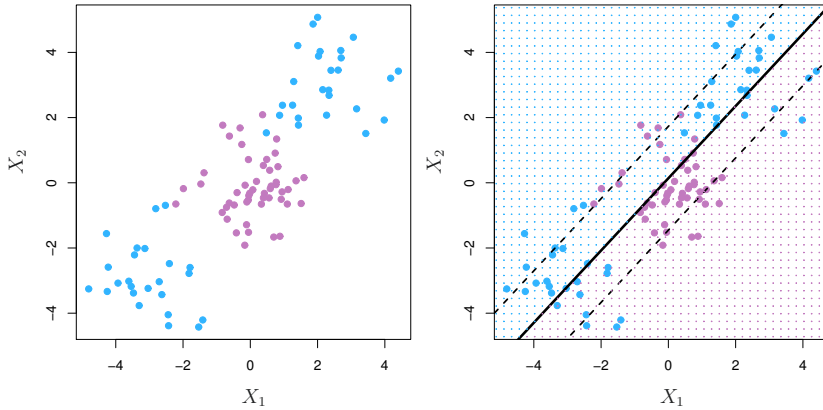
The fact that only support vectors affect the classifier is in line with our previous assertion that  $C$  controls the bias-variance trade-off of the support vector classifier. When the tuning parameter  $C$  is large, then the margin is wide, many observations violate the margin, and so there are many support vectors. In this case, many observations are involved in determining the hyperplane. The top left panel in Figure 9.7 illustrates this setting: this classifier has low variance (since many observations are support vectors) but potentially high bias. In contrast, if  $C$  is small, then there will be fewer support vectors and hence the resulting classifier will have low bias but high variance. The bottom right panel in Figure 9.7 illustrates this setting, with only eight support vectors.

The fact that the support vector classifier's decision rule is based only on a potentially small subset of the training observations (the support vectors) means that it is quite robust to the behavior of observations that are far away from the hyperplane. This property is distinct from some of the other classification methods that we have seen in preceding chapters, such as linear discriminant analysis. Recall that the LDA classification rule depends on the mean of *all* of the observations within each class, as well as



**FIGURE 9.7.** A support vector classifier was fit using four different values of the tuning parameter  $C$  in (9.12)–(9.15). The largest value of  $C$  was used in the top left panel, and smaller values were used in the top right, bottom left, and bottom right panels. When  $C$  is large, then there is a high tolerance for observations being on the wrong side of the margin, and so the margin will be large. As  $C$  decreases, the tolerance for observations being on the wrong side of the margin decreases, and the margin narrows.

the within-class covariance matrix computed using *all* of the observations. In contrast, logistic regression, unlike LDA, has very low sensitivity to observations far from the decision boundary. In fact we will see in Section 9.5 that the support vector classifier and logistic regression are closely related.



**FIGURE 9.8.** Left: The observations fall into two classes, with a non-linear boundary between them. Right: The support vector classifier seeks a linear boundary, and consequently performs very poorly.

## 9.3 Support Vector Machines

We first discuss a general mechanism for converting a linear classifier into one that produces non-linear decision boundaries. We then introduce the support vector machine, which does this in an automatic way.

### 9.3.1 Classification with Non-Linear Decision Boundaries

The support vector classifier is a natural approach for classification in the two-class setting, if the boundary between the two classes is linear. However, in practice we are sometimes faced with non-linear class boundaries. For instance, consider the data in the left-hand panel of Figure 9.8. It is clear that a support vector classifier or any linear classifier will perform poorly here. Indeed, the support vector classifier shown in the right-hand panel of Figure 9.8 is useless here.

In Chapter 7, we are faced with an analogous situation. We see there that the performance of linear regression can suffer when there is a non-linear relationship between the predictors and the outcome. In that case, we consider enlarging the feature space using functions of the predictors, such as quadratic and cubic terms, in order to address this non-linearity. In the case of the support vector classifier, we could address the problem of possibly non-linear boundaries between classes in a similar way, by enlarging the feature space using quadratic, cubic, and even higher-order polynomial functions of the predictors. For instance, rather than fitting a support vector classifier using  $p$  features

$$X_1, X_2, \dots, X_p,$$

we could instead fit a support vector classifier using  $2p$  features

$$X_1, X_1^2, X_2, X_2^2, \dots, X_p, X_p^2.$$

Then (9.12)–(9.15) would become

$$\begin{aligned} & \underset{\beta_0, \beta_{11}, \beta_{12}, \dots, \beta_{p1}, \beta_{p2}, \epsilon_1, \dots, \epsilon_n, M}{\text{maximize}} && M \\ & \text{subject to } y_i \left( \beta_0 + \sum_{j=1}^p \beta_{j1} x_{ij} + \sum_{j=1}^p \beta_{j2} x_{ij}^2 \right) \geq M(1 - \epsilon_i), \\ & \sum_{i=1}^n \epsilon_i \leq C, \quad \epsilon_i \geq 0, \quad \sum_{j=1}^p \sum_{k=1}^2 \beta_{jk}^2 = 1. \end{aligned} \tag{9.16}$$

Why does this lead to a non-linear decision boundary? In the enlarged feature space, the decision boundary that results from (9.16) is in fact linear. But in the original feature space, the decision boundary is of the form  $q(x) = 0$ , where  $q$  is a quadratic polynomial, and its solutions are generally non-linear. One might additionally want to enlarge the feature space with higher-order polynomial terms, or with interaction terms of the form  $X_j X_{j'}$  for  $j \neq j'$ . Alternatively, other functions of the predictors could be considered rather than polynomials. It is not hard to see that there are many possible ways to enlarge the feature space, and that unless we are careful, we could end up with a huge number of features. Then computations would become unmanageable. The support vector machine, which we present next, allows us to enlarge the feature space used by the support vector classifier in a way that leads to efficient computations.

### 9.3.2 The Support Vector Machine

The *support vector machine* (SVM) is an extension of the support vector classifier that results from enlarging the feature space in a specific way, using *kernels*. We will now discuss this extension, the details of which are somewhat complex and beyond the scope of this book. However, the main idea is described in Section 9.3.1: we may want to enlarge our feature space in order to accommodate a non-linear boundary between the classes. The kernel approach that we describe here is simply an efficient computational approach for enacting this idea.

support  
vector  
machine  
kernel

We have not discussed exactly how the support vector classifier is computed because the details become somewhat technical. However, it turns out that the solution to the support vector classifier problem (9.12)–(9.15) involves only the *inner products* of the observations (as opposed to the observations themselves). The inner product of two  $r$ -vectors  $a$  and  $b$  is defined as  $\langle a, b \rangle = \sum_{i=1}^r a_i b_i$ . Thus the inner product of two observations

$x_i, x_{i'}$  is given by

$$\langle x_i, x_{i'} \rangle = \sum_{j=1}^p x_{ij} x_{i'j}. \quad (9.17)$$

It can be shown that

- The linear support vector classifier can be represented as

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle x, x_i \rangle, \quad (9.18)$$

where there are  $n$  parameters  $\alpha_i$ ,  $i = 1, \dots, n$ , one per training observation.

- To estimate the parameters  $\alpha_1, \dots, \alpha_n$  and  $\beta_0$ , all we need are the  $\binom{n}{2}$  inner products  $\langle x_i, x_{i'} \rangle$  between all pairs of training observations. (The notation  $\binom{n}{2}$  means  $n(n-1)/2$ , and gives the number of pairs among a set of  $n$  items.)

Notice that in (9.18), in order to evaluate the function  $f(x)$ , we need to compute the inner product between the new point  $x$  and each of the training points  $x_i$ . However, it turns out that  $\alpha_i$  is nonzero only for the support vectors in the solution—that is, if a training observation is not a support vector, then its  $\alpha_i$  equals zero. So if  $\mathcal{S}$  is the collection of indices of these support points, we can rewrite any solution function of the form (9.18) as

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle x, x_i \rangle, \quad (9.19)$$

which typically involves far fewer terms than in (9.18).<sup>2</sup>

To summarize, in representing the linear classifier  $f(x)$ , and in computing its coefficients, all we need are inner products.

Now suppose that every time the inner product (9.17) appears in the representation (9.18), or in a calculation of the solution for the support vector classifier, we replace it with a *generalization* of the inner product of the form

$$K(x_i, x_{i'}), \quad (9.20)$$

where  $K$  is some function that we will refer to as a *kernel*. A kernel is a function that quantifies the similarity of two observations. For instance, we could simply take kernel

$$K(x_i, x_{i'}) = \sum_{j=1}^p x_{ij} x_{i'j}, \quad (9.21)$$

---

<sup>2</sup>By expanding each of the inner products in (9.19), it is easy to see that  $f(x)$  is a linear function of the coordinates of  $x$ . Doing so also establishes the correspondence between the  $\alpha_i$  and the original parameters  $\beta_j$ .

which would just give us back the support vector classifier. Equation 9.21 is known as a *linear kernel* because the support vector classifier is linear in the features; the linear kernel essentially quantifies the similarity of a pair of observations using Pearson (standard) correlation. But one could instead choose another form for (9.20). For instance, one could replace every instance of  $\sum_{j=1}^p x_{ij}x_{i'j}$  with the quantity

$$K(x_i, x_{i'}) = (1 + \sum_{j=1}^p x_{ij}x_{i'j})^d. \quad (9.22)$$

This is known as a *polynomial kernel* of degree  $d$ , where  $d$  is a positive integer. Using such a kernel with  $d > 1$ , instead of the standard linear kernel (9.21), in the support vector classifier algorithm leads to a much more flexible decision boundary. It essentially amounts to fitting a support vector classifier in a higher-dimensional space involving polynomials of degree  $d$ , rather than in the original feature space. When the support vector classifier is combined with a non-linear kernel such as (9.22), the resulting classifier is known as a support vector machine. Note that in this case the (non-linear) function has the form

polynomial  
kernel

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i). \quad (9.23)$$

The left-hand panel of Figure 9.9 shows an example of an SVM with a polynomial kernel applied to the non-linear data from Figure 9.8. The fit is a substantial improvement over the linear support vector classifier. When  $d = 1$ , then the SVM reduces to the support vector classifier seen earlier in this chapter.

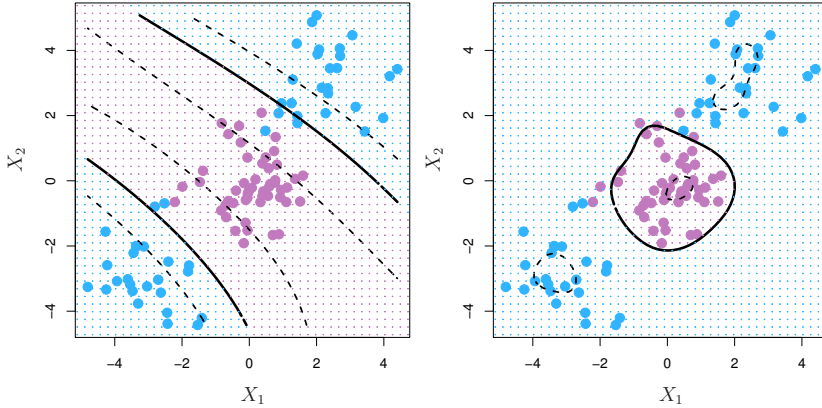
The polynomial kernel shown in (9.22) is one example of a possible non-linear kernel, but alternatives abound. Another popular choice is the *radial kernel*, which takes the form

radial kernel

$$K(x_i, x_{i'}) = \exp(-\gamma \sum_{j=1}^p (x_{ij} - x_{i'j})^2). \quad (9.24)$$

In (9.24),  $\gamma$  is a positive constant. The right-hand panel of Figure 9.9 shows an example of an SVM with a radial kernel on this non-linear data; it also does a good job in separating the two classes.

How does the radial kernel (9.24) actually work? If a given test observation  $x^* = (x_1^*, \dots, x_p^*)^T$  is far from a training observation  $x_i$  in terms of Euclidean distance, then  $\sum_{j=1}^p (x_j^* - x_{ij})^2$  will be large, and so  $K(x^*, x_i) = \exp(-\gamma \sum_{j=1}^p (x_j^* - x_{ij})^2)$  will be tiny. This means that in (9.23),  $x_i$  will play virtually no role in  $f(x^*)$ . Recall that the predicted class label for the test observation  $x^*$  is based on the sign of  $f(x^*)$ . In other words, training observations that are far from  $x^*$  will play essentially no role in the predicted class label for  $x^*$ . This means that the radial kernel has very *local*



**FIGURE 9.9.** Left: An SVM with a polynomial kernel of degree 3 is applied to the non-linear data from Figure 9.8, resulting in a far more appropriate decision rule. Right: An SVM with a radial kernel is applied. In this example, either kernel is capable of capturing the decision boundary.

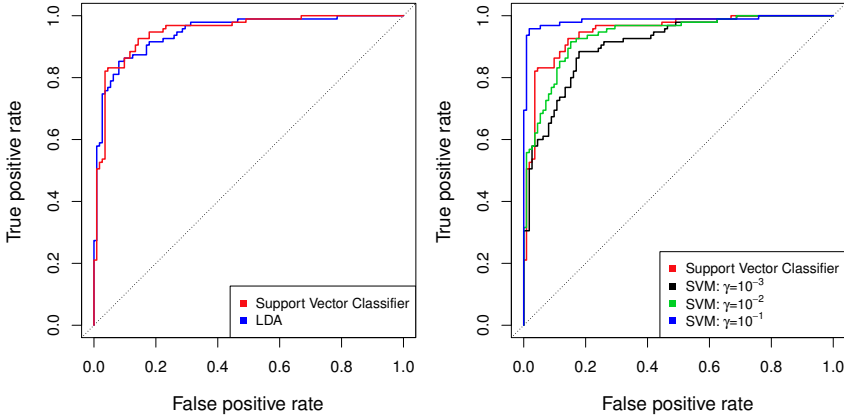
behavior, in the sense that only nearby training observations have an effect on the class label of a test observation.

What is the advantage of using a kernel rather than simply enlarging the feature space using functions of the original features, as in (9.16)? One advantage is computational, and it amounts to the fact that using kernels, one need only compute  $K(x_i, x'_i)$  for all  $\binom{n}{2}$  distinct pairs  $i, i'$ . This can be done without explicitly working in the enlarged feature space. This is important because in many applications of SVMs, the enlarged feature space is so large that computations are intractable. For some kernels, such as the radial kernel (9.24), the feature space is *implicit* and infinite-dimensional, so we could never do the computations there anyway!

### 9.3.3 An Application to the Heart Disease Data

In Chapter 8 we apply decision trees and related methods to the **Heart** data. The aim is to use 13 predictors such as **Age**, **Sex**, and **Chol** in order to predict whether an individual has heart disease. We now investigate how an SVM compares to LDA on this data. After removing 6 missing observations, the data consist of 297 subjects, which we randomly split into 207 training and 90 test observations.

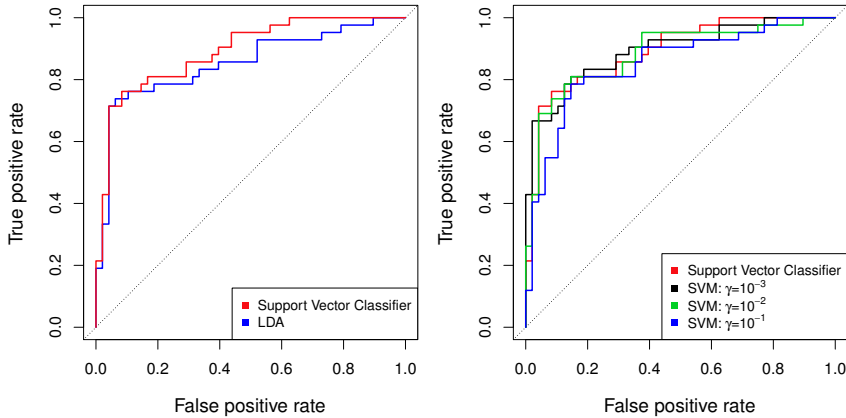
We first fit LDA and the support vector classifier to the training data. Note that the support vector classifier is equivalent to a SVM using a polynomial kernel of degree  $d = 1$ . The left-hand panel of Figure 9.10 displays ROC curves (described in Section 4.4.2) for the training set predictions for both LDA and the support vector classifier. Both classifiers compute scores of the form  $\hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \hat{\beta}_2 X_2 + \cdots + \hat{\beta}_p X_p$  for each observation.



**FIGURE 9.10.** ROC curves for the *Heart* data training set. Left: The support vector classifier and LDA are compared. Right: The support vector classifier is compared to an SVM using a radial basis kernel with  $\gamma = 10^{-3}$ ,  $10^{-2}$ , and  $10^{-1}$ .

For any given cutoff  $t$ , we classify observations into the *heart disease* or *no heart disease* categories depending on whether  $\hat{f}(X) < t$  or  $\hat{f}(X) \geq t$ . The ROC curve is obtained by forming these predictions and computing the false positive and true positive rates for a range of values of  $t$ . An optimal classifier will hug the top left corner of the ROC plot. In this instance LDA and the support vector classifier both perform well, though there is a suggestion that the support vector classifier may be slightly superior.

The right-hand panel of Figure 9.10 displays ROC curves for SVMs using a radial kernel, with various values of  $\gamma$ . As  $\gamma$  increases and the fit becomes more non-linear, the ROC curves improve. Using  $\gamma = 10^{-1}$  appears to give an almost perfect ROC curve. However, these curves represent training error rates, which can be misleading in terms of performance on new test data. Figure 9.11 displays ROC curves computed on the 90 test observations. We observe some differences from the training ROC curves. In the left-hand panel of Figure 9.11, the support vector classifier appears to have a small advantage over LDA (although these differences are not statistically significant). In the right-hand panel, the SVM using  $\gamma = 10^{-1}$ , which showed the best results on the training data, produces the worst estimates on the test data. This is once again evidence that while a more flexible method will often produce lower training error rates, this does not necessarily lead to improved performance on test data. The SVMs with  $\gamma = 10^{-2}$  and  $\gamma = 10^{-3}$  perform comparably to the support vector classifier, and all three outperform the SVM with  $\gamma = 10^{-1}$ .



**FIGURE 9.11.** ROC curves for the test set of the **Heart** data. Left: The support vector classifier and LDA are compared. Right: The support vector classifier is compared to an SVM using a radial basis kernel with  $\gamma = 10^{-3}$ ,  $10^{-2}$ , and  $10^{-1}$ .

## 9.4 SVMs with More than Two Classes

So far, our discussion has been limited to the case of binary classification: that is, classification in the two-class setting. How can we extend SVMs to the more general case where we have some arbitrary number of classes? It turns out that the concept of separating hyperplanes upon which SVMs are based does not lend itself naturally to more than two classes. Though a number of proposals for extending SVMs to the  $K$ -class case have been made, the two most popular are the *one-versus-one* and *one-versus-all* approaches. We briefly discuss those two approaches here.

### 9.4.1 One-Versus-One Classification

Suppose that we would like to perform classification using SVMs, and there are  $K > 2$  classes. A *one-versus-one* or *all-pairs* approach constructs  $\binom{K}{2}$  SVMs, each of which compares a pair of classes. For example, one such SVM might compare the  $k$ th class, coded as  $+1$ , to the  $k'$ th class, coded as  $-1$ . We classify a test observation using each of the  $\binom{K}{2}$  classifiers, and we tally the number of times that the test observation is assigned to each of the  $K$  classes. The final classification is performed by assigning the test observation to the class to which it was most frequently assigned in these  $\binom{K}{2}$  pairwise classifications.

### 9.4.2 One-Versus-All Classification

The *one-versus-all* approach is an alternative procedure for applying SVMs in the case of  $K > 2$  classes. We fit  $K$  SVMs, each time comparing one of

# Learning Important Features Through Propagating Activation Differences

Avanti Shrikumar<sup>1</sup> Peyton Greenside<sup>1</sup> Anshul Kundaje<sup>1</sup>

## Abstract

The purported “black box” nature of neural networks is a barrier to adoption in applications where interpretability is essential. Here we present DeepLIFT (Deep Learning Important FeaTures), a method for decomposing the output prediction of a neural network on a specific input by backpropagating the contributions of all neurons in the network to every feature of the input. DeepLIFT compares the activation of each neuron to its ‘reference activation’ and assigns contribution scores according to the difference. By optionally giving separate consideration to positive and negative contributions, DeepLIFT can also reveal dependencies which are missed by other approaches. Scores can be computed efficiently in a single backward pass. We apply DeepLIFT to models trained on MNIST and simulated genomic data, and show significant advantages over gradient-based methods. Video tutorial: <http://goo.gl/qKb7pL>, ICML slides: [bit.ly/deeplifticmlslides](http://bit.ly/deeplifticmlslides), ICML talk: <https://vimeo.com/238275076>, code: <http://goo.gl/RM8jvH>.

## 1. Introduction

As neural networks become increasingly popular, their black box reputation is a barrier to adoption when interpretability is paramount. Here, we present DeepLIFT (Deep Learning Important FeaTures), a novel algorithm to assign importance score to the inputs for a given output. Our approach is unique in two regards. First, it frames the question of importance in terms of differences from a ‘reference’ state, where the ‘reference’ is chosen according to the problem at hand. In contrast to most gradient-based methods, using a difference-from-reference allows DeepLIFT to propagate an importance signal even in situations where the gradient is zero and avoids artifacts caused by discontinuities in the gradient. Second, by optionally giving separate consideration to the effects of posi-

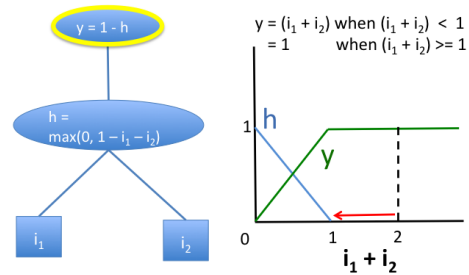
tive and negative contributions at nonlinearities, DeepLIFT can reveal dependencies missed by other approaches. As DeepLIFT scores are computed using a backpropagation-like algorithm, they can be obtained efficiently in a single backward pass after a prediction has been made.

## 2. Previous Work

This section provides a review of existing approaches to assign importance scores for a given task and input example.

### 2.1. Perturbation-Based Forward Propagation Approaches

These approaches make perturbations to individual inputs or neurons and observe the impact on later neurons in the network. Zeiler & Fergus (Zeiler & Fergus, 2013) occluded different segments of an input image and visualized the change in the activations of later layers. “In-silico mutagenesis” (Zhou & Troyanskaya, 2015) introduced virtual mutations at individual positions in a genomic sequence and quantified the their impact on the output. Zintgraf et al. (Zintgraf et al., 2017) proposed a clever strategy for analyzing the difference in a prediction after marginalizing over each input patch. However, such methods can be computationally inefficient as each perturbation requires a separate forward propagation through the network. They may also underestimate the importance of features that have saturated their contribution to the output (Fig. 1).



**Figure 1. Perturbation-based approaches and gradient-based approaches fail to model saturation.** Illustrated is a simple network exhibiting saturation in the signal from its inputs. At the point where  $i_1 = 1$  and  $i_2 = 1$ , perturbing either  $i_1$  or  $i_2$  to 0 will not produce a change in the output. Note that the gradient of the output w.r.t the inputs is also zero when  $i_1 + i_2 > 1$ .

<sup>1</sup>Stanford University, Stanford, California, USA. Correspondence: A Kundaje <akundaje@stanford.edu>.

## 2.2. Backpropagation-Based Approaches

Unlike perturbation methods, backpropagation approaches propagate an importance signal from an output neuron backwards through the layers to the input in one pass, making them efficient. DeepLIFT is one such approach.

### 2.2.1. GRADIENTS, DECONVOLUTIONAL NETWORKS AND GUIDED BACKPROPAGATION

Simonyan et al. (Simonyan et al., 2013) proposed using the gradient of the output w.r.t. pixels of an input image to compute a “saliency map” of the image in the context of image classification tasks. The authors showed that this was similar to deconvolutional networks (Zeiler & Fergus, 2013) except for the handling of the nonlinearity at rectified linear units (ReLU). When backpropagating importance using gradients, the gradient coming into a ReLU during the backward pass is zero’d out if the input to the ReLU during the forward pass is negative. By contrast, when backpropagating an importance signal in deconvolutional networks, the importance signal coming into a ReLU during the backward pass is zero’d out if and only if it is negative, with no regard to sign of the input to the ReLU during the forward pass. Springenberg et al., (Springenberg et al., 2014) combined these two approaches into Guided Backpropagation, which zero’s out the importance signal at a ReLU if either the input to the ReLU during the forward pass is negative or the importance signal during the backward pass is negative. Guided Backpropagation can be thought of as equivalent to computing gradients, with the caveat that any gradients that become negative during the backward pass are discarded at ReLUs. Due to the zero-ing out of negative gradients, both guided backpropagation and deconvolutional networks can fail to highlight inputs that contribute negatively to the output. Additionally, none of the three approaches would address the saturation problem illustrated in Fig. 1, as the gradient of  $y$  w.r.t.  $h$  is negative (causing Guided Backprop and deconvolutional networks to assign zero importance), and the gradient of  $h$  w.r.t both  $i_1$  and  $i_2$  is zero when  $i_1 + i_2 > 1$  (causing both gradients and Guided Backprop to be zero). Discontinuities in the gradients can also cause undesirable artifacts (Fig. 2).

### 2.2.2. LAYERWISE RELEVANCE PROPAGATION AND GRADIENT $\times$ INPUT

Bach et al. (Bach et al., 2015) proposed an approach for propagating importance scores called Layerwise Relevance Propagation (LRP). Shrikumar et al. and Kindermans et al. (Shrikumar et al., 2016; Kindermans et al., 2016) showed that absent modifications to deal with numerical stability, the LRP rules for ReLU networks were equivalent within a scaling factor to an elementwise product between the saliency maps of Simonyan et al. and the input (in other

words, gradient  $\times$  input). In our experiments, we compare DeepLIFT to gradient  $\times$  input as the latter is easily implemented on a GPU, whereas (at the time of writing) LRP did not have a GPU implementation available to our knowledge.

While gradient  $\times$  input is often preferable to gradients alone as it leverages the sign and strength of the input, it still does not address the saturation problem in Fig. 1 or the thresholding artifact in Fig. 2.

### 2.2.3. INTEGRATED GRADIENTS

Instead of computing the gradients at only the current value of the input, one can integrate the gradients as the inputs are scaled up from some starting value (eg: all zeros) to their current value (Sundararajan et al., 2016). This addresses the saturation and thresholding problems of Fig. 1 and Fig. 2, but numerically obtaining high-quality integrals adds computational overhead. Further, this approach can still give misleading results (see Section 3.4.3).

## 2.3. Grad-CAM and Guided CAM

Grad-CAM (Selvaraju et al., 2016) computes a coarse-grained feature-importance map by associating the feature maps in the final convolutional layer with particular classes based on the gradients of each class w.r.t. each feature map, and then using the weighted activations of the feature maps as an indication of which inputs are most important. To obtain more fine-grained feature importance, the authors proposed performing an elementwise product between the scores obtained from Grad-CAM and the scores obtained from Guided Backpropagation, termed Guided Grad-CAM. However, this strategy inherits the limitations of Guided Backpropagation caused by zero-ing out negative gradients during backpropagation. It is also specific to convolutional neural networks.

## 3. The DeepLIFT Method

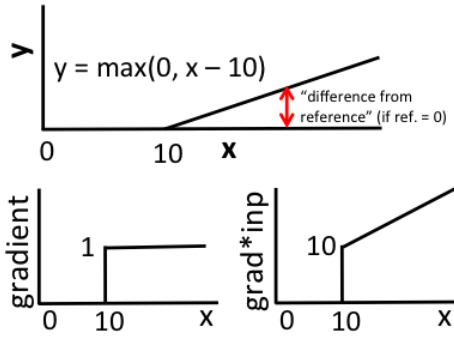
### 3.1. The DeepLIFT Philosophy

DeepLIFT explains the difference in output from some ‘reference’ output in terms of the difference of the input from some ‘reference’ input. The ‘reference’ input represents some default or ‘neutral’ input that is chosen according to what is appropriate for the problem at hand (see Section 3.3 for more details). Formally, let  $t$  represent some target output neuron of interest and let  $x_1, x_2, \dots, x_n$  represent some neurons in some intermediate layer or set of layers that are necessary and sufficient to compute  $t$ . Let  $t^0$  represent the reference activation of  $t$ . We define the quantity  $\Delta t$  to be the difference-from-reference, that is  $\Delta t = t - t^0$ . DeepLIFT assigns contribution scores  $C_{\Delta x_i \Delta t}$  to  $\Delta x_i$  s.t.:

$$\sum_{i=1}^n C_{\Delta x_i \Delta t} = \Delta t \quad (1)$$

We call **Eq. 1** the **summation-to-delta** property.  $C_{\Delta x_i \Delta t}$  can be thought of as the amount of difference-from-reference in  $t$  that is attributed to or ‘blamed’ on the difference-from-reference of  $x_i$ . Note that when a neuron’s transfer function is well-behaved, the output is locally linear in its inputs, providing additional motivation for **Eq. 1**.

$C_{\Delta x_i \Delta t}$  can be non-zero even when  $\frac{\partial t}{\partial x_i}$  is zero. This allows DeepLIFT to address a fundamental limitation of gradients because, as illustrated in **Fig. 1**, a neuron can be signaling meaningful information even in the regime where its gradient is zero. Another drawback of gradients addressed by DeepLIFT is illustrated in **Fig. 2**, where the discontinuous nature of gradients causes sudden jumps in the importance score over infinitesimal changes in the input. By contrast, the difference-from-reference is continuous, allowing DeepLIFT to avoid discontinuities caused by bias terms.



**Figure 2. Discontinuous gradients can produce misleading importance scores.** Response of a single rectified linear unit with a bias of  $-10$ . Both gradient and gradient  $\times$  input have a discontinuity at  $x = 10$ ; at  $x = 10 + \epsilon$ , gradient  $\times$  input assigns a contribution of  $10 + \epsilon$  to  $x$  and  $-10$  to the bias term ( $\epsilon$  is a small positive number). When  $x < 10$ , contributions on  $x$  and the bias term are both 0. By contrast, the difference-from-reference (red arrow, top figure) gives a continuous increase in the contribution score.

## 3.2. Multipliers and the Chain Rule

### 3.2.1. DEFINITION OF MULTIPLIERS

For a given input neuron  $x$  with difference-from-reference  $\Delta x$ , and target neuron  $t$  with difference-from-reference  $\Delta t$  that we wish to compute the contribution to, we define the multiplier  $m_{\Delta x \Delta t}$  as:

$$m_{\Delta x \Delta t} = \frac{C_{\Delta x \Delta t}}{\Delta x} \quad (2)$$

In other words, the multiplier  $m_{\Delta x \Delta t}$  is the contribution of  $\Delta x$  to  $\Delta t$  divided by  $\Delta x$ . Note the close analogy to the

idea of partial derivatives: the partial derivative  $\frac{\partial t}{\partial x}$  is the infinitesimal change in  $t$  caused by an infinitesimal change in  $x$ , divided by the infinitesimal change in  $x$ . The multiplier is similar in spirit to a partial derivative, but over finite differences instead of infinitesimal ones.

### 3.2.2. THE CHAIN RULE FOR MULTIPLIERS

Assume we have an input layer with neurons  $x_1, \dots, x_n$ , a hidden layer with neurons  $y_1, \dots, y_n$ , and some target output neuron  $t$ . Given values for  $m_{\Delta x_i \Delta y_j}$  and  $m_{\Delta y_j \Delta t}$ , the following definition of  $m_{\Delta x_i \Delta t}$  is consistent with the summation-to-delta property in **Eq. 1** (see **Appendix A** for the proof):

$$m_{\Delta x_i \Delta t} = \sum_j m_{\Delta x_i \Delta y_j} m_{\Delta y_j \Delta t} \quad (3)$$

We refer to **Eq. 3** as the **chain rule for multipliers**. Given the multipliers for each neuron to its immediate successors, we can compute the multipliers for any neuron to a given target neuron efficiently via backpropagation - analogous to how the chain rule for partial derivatives allows us to compute the gradient w.r.t. the output via backpropagation.

## 3.3. Defining the Reference

When formulating the DeepLIFT rules described in **Section 3.5**, we assume that the reference of a neuron is its activation on the reference input. Formally, say we have a neuron  $y$  with inputs  $x_1, x_2, \dots$  such that  $y = f(x_1, x_2, \dots)$ . Given the reference activations  $x_1^0, x_2^0, \dots$  of the inputs, we can calculate the reference activation  $y^0$  of the output as:

$$y^0 = f(x_1^0, x_2^0, \dots) \quad (4)$$

i.e. references for all neurons can be found by choosing a reference input and propagating activations through the net.

The choice of a reference input is critical for obtaining insightful results from DeepLIFT. In practice, choosing a good reference would rely on domain-specific knowledge, and in some cases it may be best to compute DeepLIFT scores against multiple different references. As a guiding principle, we can ask ourselves ‘‘what am I interested in measuring differences against?’’. For MNIST, we use a reference input of all-zeros as this is the background of the images. For the binary classification tasks on DNA sequence inputs (strings over the alphabet  $\{A, C, G, T\}$ ), we obtained sensible results using either a reference input containing the expected frequencies of ACGT in the background (**Fig. 5**), or by averaging the results over multiple reference inputs for each sequence that are generated by shuffling each original sequence (**Appendix J**). For CIFAR10 data, we found that using a blurred version of the original image as the

reference highlighted outlines of key objects, while an all-zeros reference highlighted hard-to-interpret pixels in the background (**Appendix L**).

It is important to note that  $\text{gradient} \times \text{input}$  implicitly uses a reference of all-zeros (it is equivalent to a first-order Taylor approximation of  $\text{gradient} \times \Delta \text{input}$  where  $\Delta$  is measured w.r.t. an input of zeros). Similarly, integrated gradients (**Section 2.2.3**) requires the user to specify a starting point for the integral, which is conceptually similar to specifying a reference for DeepLIFT. While Guided Backprop and pure gradients don't use a reference, we argue that this is a limitation as these methods only describe the local behaviour of the output at the specific input value, without considering how the output behaves over a range of inputs.

### 3.4. Separating Positive and Negative Contributions

We will see in **Section 3.5.3** that, in some situations, it is essential to treat positive and negative contributions differently. To do this, for every neuron  $y$ , we will introduce  $\Delta y^+$  and  $\Delta y^-$  to represent the positive and negative components of  $\Delta y$ , such that:

$$\begin{aligned}\Delta y &= \Delta y^+ + \Delta y^- \\ C_{\Delta y \Delta t} &= C_{\Delta y^+ \Delta t} + C_{\Delta y^- \Delta t}\end{aligned}$$

For linear neurons,  $\Delta y^+$  and  $\Delta y^-$  are found by writing  $\Delta y$  as a sum of terms involving its inputs  $\Delta x_i$  and grouping positive and negative terms together. The importance of this will become apparent when applying the RevealCancel rule (**Section 3.5.3**), where for a given target neuron  $t$  we may find that  $m_{\Delta y^+ \Delta t}$  and  $m_{\Delta y^- \Delta t}$  differ. However, when applying only the Linear or Rescale rules (**Section 3.5.1** and **Section 3.5.2**),  $m_{\Delta y \Delta t} = m_{\Delta y^+ \Delta t} = m_{\Delta y^- \Delta t}$ .

### 3.5. Rules for Assigning Contribution Scores

We present the rules for assigning contribution scores for each neuron to its immediate inputs. In conjunction with the chain rule for multipliers (**Section 3.2**), these rules can be used to find the contributions of any input (not just the immediate inputs) to a target output via backpropagation.

#### 3.5.1. THE LINEAR RULE

This applies to Dense and Convolutional layers (excluding nonlinearities). Let  $y$  be a linear function of its inputs  $x_i$  such that  $y = b + \sum_i w_i x_i$ . We have  $\Delta y = \sum_i w_i \Delta x_i$ . We define the positive and negative parts of  $\Delta y$  as:

$$\begin{aligned}\Delta y^+ &= \sum_i 1\{w_i \Delta x_i > 0\} w_i \Delta x_i \\ &= \sum_i 1\{w_i \Delta x_i > 0\} w_i (\Delta x_i^+ + \Delta x_i^-)\end{aligned}$$

$$\begin{aligned}\Delta y^- &= \sum_i 1\{w_i \Delta x_i < 0\} w_i \Delta x_i \\ &= \sum_i 1\{w_i \Delta x_i < 0\} w_i (\Delta x_i^+ + \Delta x_i^-)\end{aligned}$$

Which leads to the following choice for the contributions:

$$\begin{aligned}C_{\Delta x_i^+ \Delta y^+} &= 1\{w_i \Delta x_i > 0\} w_i \Delta x_i^+ \\ C_{\Delta x_i^- \Delta y^+} &= 1\{w_i \Delta x_i > 0\} w_i \Delta x_i^- \\ C_{\Delta x_i^+ \Delta y^-} &= 1\{w_i \Delta x_i < 0\} w_i \Delta x_i^+ \\ C_{\Delta x_i^- \Delta y^-} &= 1\{w_i \Delta x_i < 0\} w_i \Delta x_i^-\end{aligned}$$

We can then find multipliers using the definition in **Section 3.2.1**, which gives  $m_{\Delta x_i^+ \Delta y^+} = m_{\Delta x_i^- \Delta y^+} = 1\{w_i \Delta x_i > 0\} w_i$  and  $m_{\Delta x_i^+ \Delta y^-} = m_{\Delta x_i^- \Delta y^-} = 1\{w_i \Delta x_i < 0\} w_i$ .

What about when  $\Delta x_i = 0$ ? While setting multipliers to 0 in this case would be consistent with summation-to-delta, it is possible that  $\Delta x_i^+$  and  $\Delta x_i^-$  are nonzero (and cancel each other out), in which case setting the multiplier to 0 would fail to propagate importance to them. To avoid this, we set  $m_{\Delta x_i^+ \Delta y^+} = m_{\Delta x_i^+ \Delta y^-} = 0.5 w_i$  when  $\Delta x_i$  is 0 (similarly for  $\Delta x_i^-$ ). See **Appendix B** for how to compute these multipliers using standard neural network ops.

#### 3.5.2. THE RESCALE RULE

This rule applies to nonlinear transformations that take a single input, such as the ReLU, tanh or sigmoid operations. Let neuron  $y$  be a nonlinear transformation of its input  $x$  such that  $y = f(x)$ . Because  $y$  has only one input, we have by summation-to-delta that  $C_{\Delta x \Delta y} = \Delta y$ , and consequently  $m_{\Delta x \Delta y} = \frac{\Delta y}{\Delta x}$ . For the Rescale rule, we set  $\Delta y^+$  and  $\Delta y^-$  proportional to  $\Delta x^+$  and  $\Delta x^-$  as follows:

$$\begin{aligned}\Delta y^+ &= \frac{\Delta y}{\Delta x} \Delta x^+ = C_{\Delta x^+ \Delta y^+} \\ \Delta y^- &= \frac{\Delta y}{\Delta x} \Delta x^- = C_{\Delta x^- \Delta y^-}\end{aligned}$$

Based on this, we get:

$$m_{\Delta x^+ \Delta y^+} = m_{\Delta x^- \Delta y^-} = m_{\Delta x \Delta y} = \frac{\Delta y}{\Delta x}$$

In the case where  $x \rightarrow x^0$ , we have  $\Delta x \rightarrow 0$  and  $\Delta y \rightarrow 0$ . The definition of the multiplier approaches the derivative, i.e.  $m_{\Delta x \Delta y} \rightarrow \frac{dy}{dx}$ , where the  $\frac{dy}{dx}$  is evaluated at  $x = x^0$ . We can thus use the gradient instead of the multiplier when  $x$  is close to its reference to avoid numerical instability issues caused by having a small denominator.

Note that the Rescale rule addresses both the saturation and the thresholding problems illustrated in **Fig. 1** and **Fig. 2**. In the case of **Fig. 1**, if  $i_1^0 = i_2^0 = 0$ , then at  $i_1 + i_2 > 1$  we have  $\Delta h = -1$  and  $\Delta y = 1$ , giving

$m_{\Delta h \Delta y} = \frac{\Delta y}{\Delta h} = -1$  even though  $\frac{dy}{dh} = 0$  (in other words, using difference-from-reference allows information to flow even when the gradient is zero). In the case of **Fig. 2**, assuming  $x^0 = y^0 = 0$ , at  $x = 10 + \epsilon$  we have  $\Delta y = \epsilon$ , giving  $m_{\Delta x \Delta y} = \frac{\epsilon}{10+\epsilon}$  and  $C_{\Delta x \Delta y} = \Delta x \times m_{\Delta x \Delta y} = \epsilon$ . By contrast, gradient $\times$ input assigns a contribution of  $10 + \epsilon$  to  $x$  and  $-10$  to the bias term (DeepLIFT never assigns importance to bias terms).

As revealed in previous work (Lundberg & Lee, 2016), there is a connection between DeepLIFT and Shapely values. Briefly, the Shapely values measure the average marginal effect of including an input over all possible orderings in which inputs can be included. If we define “including” an input as setting it to its actual value instead of its reference value, DeepLIFT can be thought of as a fast approximation of the Shapely values. At the time, Lundberg & Lee cited a preprint of DeepLIFT which described only the Linear and Rescale rules with no separate treatment of positive and negative contributions.

### 3.5.3. AN IMPROVED APPROXIMATION OF THE SHAPELY VALUES: THE REVEALCANCEL RULE

While the Rescale rule improves upon simply using gradients, there are still some situations where it can provide misleading results. Consider the  $\min(i_1, i_2)$  operation depicted in **Fig. 3**, with reference values of  $i_1 = 0$  and  $i_2 = 0$ . Using the Rescale rule, all importance would be assigned either to  $i_1$  or to  $i_2$  (whichever is smaller). This can obscure the fact that both inputs are relevant for the min operation.

To understand why this occurs, consider the case when  $i_1 > i_2$ . We have  $h_1 = (i_1 - i_2) > 0$  and  $h_2 = \max(0, h_1) = h_1$ . By the Linear rule, we calculate that  $C_{\Delta i_1 \Delta h_1} = i_1$  and  $C_{\Delta i_2 \Delta h_1} = -i_2$ . By the Rescale rule, the multiplier  $m_{\Delta h_1 \Delta h_2}$  is  $\frac{\Delta h_2}{\Delta h_1} = 1$ , and thus  $C_{\Delta i_1 \Delta h_2} = m_{\Delta h_1 \Delta h_2} C_{\Delta i_1 \Delta h_1} = i_1$  and  $C_{\Delta i_2 \Delta h_2} = m_{\Delta h_1 \Delta h_2} C_{\Delta i_2 \Delta h_1} = -i_2$ . The total contribution of  $i_1$  to the output  $o$  becomes  $(i_1 - C_{\Delta i_1 \Delta h_2}) = (i_1 - i_1) = 0$ , and the total contribution of  $i_2$  to  $o$  is  $-C_{\Delta i_2 \Delta h_2} = i_2$ . This calculation is misleading as it discounts the fact that  $C_{\Delta i_2 \Delta h_2}$  would be 0 if  $i_1$  were 0 - in other words, it ignores a dependency induced between  $i_1$  and  $i_2$  that comes from  $i_2$  canceling out  $i_1$  in the nonlinear neuron  $h_2$ . A similar failure occurs when  $i_1 < i_2$ ; the Rescale rule results in  $C_{\Delta i_1 \Delta o} = i_1$  and  $C_{\Delta i_2 \Delta o} = 0$ . Note that gradients, gradient $\times$ input, Guided Backpropagation and integrated gradients would also assign all importance to either  $i_1$  or  $i_2$ , because for any given input the gradient is zero for one of  $i_1$  or  $i_2$  (see **Appendix C** for a detailed calculation).

One way to address this is by treating the positive and negative contributions separately. We again consider the nonlinear neuron  $y = f(x)$ . Instead of assuming that  $\Delta y^+$  and  $\Delta y^-$  are proportional to  $\Delta x^+$  and  $\Delta x^-$  and that

$m_{\Delta x^+ \Delta y^+} = m_{\Delta x^- \Delta y^-} = m_{\Delta x \Delta y}$  (as is done for the Rescale rule), we define them as follows:

$$\begin{aligned} \Delta y^+ &= \frac{1}{2} (f(x^0 + \Delta x^+) - f(x^0)) \\ &\quad + \frac{1}{2} (f(x^0 + \Delta x^- + \Delta x^+) - f(x^0 + \Delta x^-)) \\ \Delta y^- &= \frac{1}{2} (f(x^0 + \Delta x^-) - f(x^0)) \\ &\quad + \frac{1}{2} (f(x^0 + \Delta x^+ + \Delta x^-) - f(x^0 + \Delta x^+)) \\ m_{\Delta x^+ \Delta y^+} &= \frac{C_{\Delta x^+ y^+}}{\Delta x^+} = \frac{\Delta y^+}{\Delta x^+}; m_{\Delta x^- \Delta y^-} = \frac{\Delta y^-}{\Delta x^-} \end{aligned}$$

In other words, we set  $\Delta y^+$  to the average impact of  $\Delta x^+$  after no terms have been added and after  $\Delta x^-$  has been added, and we set  $\Delta y^-$  to the average impact of  $\Delta x^-$  after no terms have been added and after  $\Delta x^+$  has been added. This can be thought of as the Shapely values of  $\Delta x^+$  and  $\Delta x^-$  contributing to  $y$ .

By considering the impact of the positive terms in the absence of negative terms, and the impact of negative terms in the absence of positive terms, we alleviate some of the issues that arise from positive and negative terms canceling each other out. In the case of **Fig. 3**, RevealCancel would assign a contribution of  $0.5 \min(i_1, i_2)$  to both inputs (see **Appendix C** for a detailed calculation).

While the RevealCancel rule also avoids the saturation and thresholding pitfalls illustrated in **Fig. 1** and **Fig. 2**, there are some circumstances where we might prefer to use the Rescale rule. Specifically, consider a thresholded ReLU where  $\Delta y > 0$  iff  $\Delta x \geq b$ . If  $\Delta x < b$  merely indicates noise, we would want to assign contributions of 0 to both  $\Delta x^+$  and  $\Delta x^-$  (as done by the Rescale rule) to mitigate the noise. RevealCancel may assign nonzero contributions by considering  $\Delta x^+$  in the absence of  $\Delta x^-$  and vice versa.

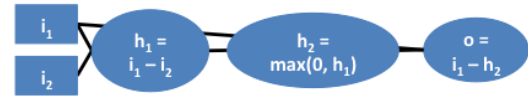


Figure 3. Network computing  $o = \min(i_1, i_2)$ . Assume  $i_1^0 = i_2^0 = 0$ . When  $i_1 < i_2$  then  $\frac{dy}{di_2} = 0$ , and when  $i_2 < i_1$  then  $\frac{dy}{di_1} = 0$ . Using any of the backpropagation approaches described in **Section 2.2** would result in importance assigned either exclusively to  $i_1$  or  $i_2$ . With the RevealCancel rule, the net assigns  $0.5 \min(i_1, i_2)$  importance to both inputs.

### 3.6. Choice of Target Layer

In the case of softmax or sigmoid outputs, we may prefer to compute contributions to the linear layer preceding the final nonlinearity rather than the final nonlinearity itself. This would be to avoid an attenuation caused by the

summation-to-delta property described in **Section 3.1**. For example, consider a sigmoid output  $o = \sigma(y)$ , where  $y$  is the logit of the sigmoid function. Assume  $y = x_1 + x_2$ , where  $x_1^0 = x_2^0 = 0$ . When  $x_1 = 50$  and  $x_2 = 0$ , the output  $o$  saturates at very close to 1 and the contributions of  $x_1$  and  $x_2$  are 0.5 and 0 respectively. However, when  $x_1 = 100$  and  $x_2 = 100$ , the output  $o$  is still very close to 1, but the contributions of  $x_1$  and  $x_2$  are now both 0.25. This can be misleading when comparing scores across different inputs because a stronger contribution to the logit would not always translate into a higher DeepLIFT score. To avoid this, we compute contributions to  $y$  rather than  $o$ .

### Adjustments for Softmax Layers

If we compute contributions to the linear layer preceding the softmax rather than the softmax output, an issue that could arise is that the final softmax output involves a normalization over all classes, but the linear layer before the softmax does not. To address this, we can normalize the contributions to the linear layer by subtracting the mean contribution to all classes. Formally, if  $n$  is the number of classes,  $C_{\Delta x \Delta c_i}$  represents the unnormalized contribution to class  $c_i$  in the linear layer and  $C'_{\Delta x \Delta c_i}$  represents the normalized contribution, we have:

$$C'_{\Delta x \Delta c_i} = C_{\Delta x \Delta c_i} - \frac{1}{n} \sum_{j=1}^n C_{\Delta x \Delta c_j} \quad (5)$$

As a justification for this normalization, we note that subtracting a fixed value from all the inputs to the softmax leaves the output of the softmax unchanged.

## 4. Results

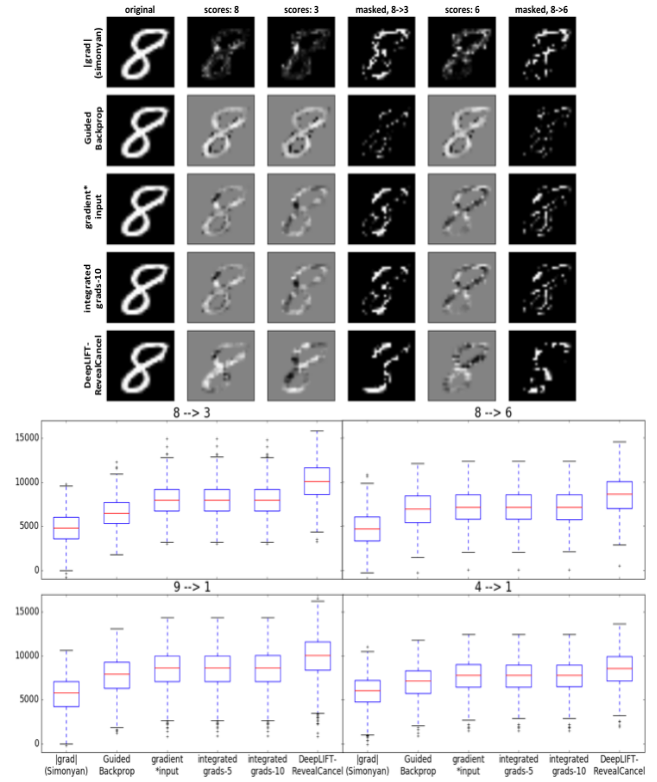
### 4.1. Digit Classification (MNIST)

We train a convolutional neural network on MNIST (LeCun et al., 1999) using Keras (Chollet, 2015) to perform digit classification and obtain 99.2% test-set accuracy. The architecture consists of two convolutional layers, followed by a fully connected layer, followed by the softmax output layer (see **Appendix D** for full details on model architecture and training). We used convolutions with stride  $> 1$  instead of pooling layers, which did not result in a drop in performance as is consistent with previous work (Springenberg et al., 2014). For DeepLIFT and integrated gradients, we used a reference input of all zeros.

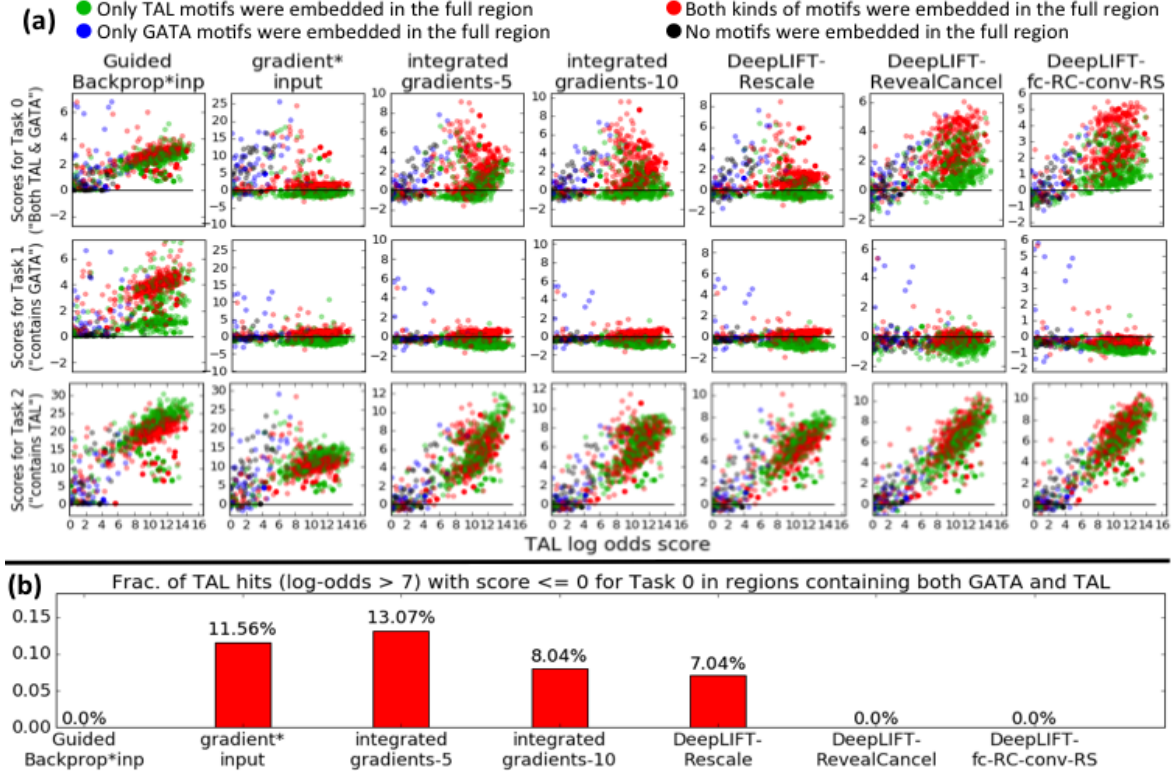
To evaluate importance scores obtained by different methods, we design the following task: given an image that originally belongs to class  $c_o$ , we identify which pixels to erase to convert the image to some target class  $c_t$ . We do this by finding  $S_{x_i \text{diff}} = S_{x_i c_o} - S_{x_i c_t}$  (where  $S_{x_i c}$  is the score for pixel  $x_i$  and class  $c$ ) and erasing up to 157 pixels (20% of the image) ranked in descending order of  $S_{x_i \text{diff}}$  for which

$S_{x_i \text{diff}} > 0$ . We then evaluate the change in the log-odds score between classes  $c_o$  and  $c_t$  for the original image and the image with the pixels erased.

As shown in **Fig. 4**, DeepLIFT with the RevealCancel rule outperformed the other backpropagation-based methods. Integrated gradients (**Section 2.2.3**) computed numerically over either 5 or 10 intervals produced results comparable to each other, suggesting that adding more intervals would not change the result. Integrated gradients also performed comparably to gradient\*input, suggesting that saturation and thresholding failure modes are not common on MNIST data. Guided Backprop discards negative gradients during backpropagation, perhaps explaining its poor performance at discriminating between classes. We also explored using the Rescale rule instead of RevealCancel on various layers and found that it degraded performance (**Appendix E**).



**Figure 4. DeepLIFT with the RevealCancel rule better identifies pixels to convert one digit to another.** Top: result of masking pixels ranked as most important for the original class (8) relative to the target class (3 or 6). Importance scores for class 8, 3 and 6 are also shown. The selected image had the highest change in log-odds scores for the 8→6 conversion using gradient\*input or integrated gradients to rank pixels. Bottom: boxplots of increase in log-odds scores of target vs. original class after the mask is applied, for 1K images belonging to the original class in the testing set. “Integrated gradients-n” refers to numerically integrating the gradients over  $n$  evenly-spaced intervals using the midpoint rule.



**Figure 5. DeepLIFT with RevealCancel gives qualitatively desirable behavior on TAL-GATA simulation.** (a) Scatter plots of importance score vs. strength of TAL1 motif match for different tasks and methods (see **Appendix G** for GATA1). For each region, top 5 motif matches are plotted. X-axes: log-odds of TAL1 motif match vs. background. Y-axes: total importance assigned to the match for specified task. Red dots are from regions where both TAL1 and GATA1 motifs were inserted during simulation; blue have GATA1 only, green have TAL1 only, black have no motifs inserted. “DeepLIFT-fc-RC-conv-RS” refers to using RevealCancel on the fully-connected layer and Rescale on the convolutional layers, which appears to reduce noise relative to using RevealCancel on all layers. (b) proportion of strong matches (log-odds > 7) to TAL1 motif in regions containing both TAL1 and GATA1 that had total score  $\leq 0$  for task 0; Guided Backprop $\times$ inp and DeepLIFT with RevealCancel have no false negatives, but Guided Backprop has false positives for Task 1 (Panel (a))

## 4.2. Classifying Regulatory DNA (Genomics)

Next, we compared the importance scoring methods when applied to classification tasks on DNA sequence inputs (strings over the alphabet  $\{A,C,G,T\}$ ). The human genome has millions of DNA sequence elements (200-1000 in length) containing specific combinations of short functional words to which regulatory proteins (RPs) bind to regulate gene activity. Each RP (e.g. GATA1) has binding affinity to specific collections of short DNA words (motifs) (e.g. GATAA and GATTA). A key problem in computational genomics is the discovery of motifs in regulatory DNA elements that give rise to distinct molecular signatures (labels) which can be measured experimentally. Here, in order to benchmark DeepLIFT and competing methods to uncover predictive patterns in DNA sequences, we design a simple simulation that captures the essence of the motif discovery problem described above.

Background DNA sequences of length 200 were generated by sampling the letters ACGT at each position with

probabilities 0.3, 0.2, 0.2 and 0.3 respectively. Motif instances were randomly sampled from previously known probabilistic motif models (See **Appendix F**) of two RPs named GATA1 and TAL1 (**Fig. 6a**) (Kheradpour & Kelis, 2014), and 0-3 instances of a given motif were inserted at random non-overlapping positions in the DNA sequences. We trained a multi-task neural network with two convolutional layers, global average pooling and one fully-connected layer on 3 binary classification tasks. Positive labeled sequences in task 1 represented “both GATA1 and TAL1 present”, task 2 represented “GATA1 present” and in task 3 represented “TAL1 present”.  $\frac{1}{4}$  of sequences had both GATA1 and TAL1 motifs (labeled 111),  $\frac{1}{4}$  had only GATA1 (labeled 010),  $\frac{1}{4}$  had only TAL1 (labeled 001), and  $\frac{1}{4}$  had no motifs (labeled 000). Details of the simulation, network architecture and predictive performance are given in **Appendix F**. For DeepLIFT and integrated gradients, we used a reference input that had the expected frequencies of ACGT at each position (i.e. we set the ACGT channel axis to 0.3, 0.2, 0.2, 0.3; see **Appendix J** for results using

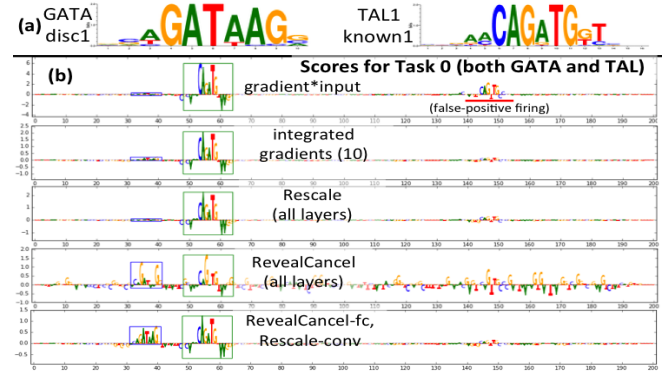
shuffled sequences as a reference). For fair comparison, this reference was also used for gradient $\times$ input and Guided Backprop $\times$ input (“input” is more accurately called  $\Delta$ input where  $\Delta$  measured w.r.t the reference). For DNA sequence inputs, we found Guided Backprop $\times$ input performed better than vanilla Guided Backprop; thus, we used the former.

Given a particular subsequence, it is possible to compute the log-odds score that the subsequence was sampled from a particular motif vs. originating from the background distribution of ACGT. To evaluate different importance-scoring methods, we found the top 5 matches (as ranked by their log-odds score) to each motif for each sequence from the test set, as well as the total importance allocated to the match by different importance-scoring methods for each task. The results are shown in **Fig. 5** (for TAL1) and **Appendix E** (for GATA1). Ideally, we expect an importance scoring method to show the following properties: (1) high scores for TAL1 motifs on task 2 and (2) low scores for TAL1 on task 1, with (3) higher scores corresponding to stronger log-odds matches; analogous pattern for GATA1 motifs (high for task 1, low for task 2); (4) high scores for both TAL1 and GATA1 motifs for task 0, with (5) higher scores on sequences containing both kinds of motifs vs. sequences containing only one kind (revealing cooperativity; corresponds to red dots lying above green dots in **Fig. 5**).

We observe Guided Backprop $\times$ input fails (2) by assigning positive importance to TAL1 on task 1 (see **Appendix H** for an example sequence). It fails property (4) by failing to identify cooperativity in task 0 (red dots overlay green dots). Both Guided Backprop $\times$ input and gradient $\times$ input show suboptimal behavior regarding property (3), in that there is a sudden increase in importance when the log-odds score is around 7, but little differentiation at higher log-odds scores (by contrast, the other methods show a more gradual increase). As a result, Guided Backprop $\times$ input and gradient $\times$ input can assign unduly high importance to weak motif matches (**Fig. 6**). This is a practical consequence of the thresholding problem from **Fig. 2**. The large discontinuous jumps in gradient also result in inflated scores (note the scale on the y-axes) relative to other methods.

We explored three versions of DeepLIFT: Rescale at all nonlinearities (DeepLIFT-Rescale), RevealCancel at all nonlinearities (DeepLIFT-RevealCancel), and Rescale at convolutional layers with RevealCancel at the fully connected layer (DeepLIFT-fc-RC-conv-RS). In contrast to the results on MNIST, we found that DeepLIFT-fc-RC-conv-RS reduced noise relative to pure RevealCancel. We think this is because of the noise-suppression property discussed in **Section 3.5.3**; if the convolutional layers act like motif detectors, the input to convolutional neurons that do not fire may just represent noise and importance should not be propagated to them (see **Fig. 6** for an example sequence).

Gradient $\times$ inp, integrated gradients and DeepLIFT-Rescale occasionally miss relevance of TAL1 for Task 0 (**Fig. 5b**), which is corrected by using RevealCancel on the fully connected layer (see example sequence in **Fig. 6**). Note that the RevealCancel scores seem to be tiered. As illustrated in **Appendix I**, this is related to having multiple instances of a given motif in a sequence (eg: when there are multiple TAL1 motifs, the importance assigned to the presence of TAL1 is distributed across all the motifs).



**Figure 6. RevealCancel highlights both TAL1 and GATA1 motifs for Task 0.** (a) PWM representations of the GATA1 motif and TAL1 motif used in the simulation (b) Scores for example sequence containing both TAL1 and GATA1 motifs. Letter height reflects the score. Blue box is location of embedded GATA1 motif, green box is location of embedded TAL1 motif. Red underline is chance occurrence of weak match to TAL1 (CAGTTG instead of CAGATG). Both TAL1 and GATA1 motifs should be highlighted for Task 0. RevealCancel on only the fully-connected layer reduces noise compared to RevealCancel on all layers.

## 5. Conclusion

We have presented DeepLIFT, a novel approach for computing importance scores based on explaining the difference of the output from some ‘reference’ output in terms of differences of the inputs from their ‘reference’ inputs. Using the difference-from-reference allows information to propagate even when the gradient is zero (**Fig. 1**), which could prove especially useful in Recurrent Neural Networks where saturating activations like sigmoid or tanh are popular. DeepLIFT avoids placing potentially misleading importance on bias terms (in contrast to gradient $\times$ input - see **Fig. 2**). By allowing separate treatment of positive and negative contributions, the DeepLIFT-RevealCancel rule can identify dependencies missed by other methods (**Fig. 3**). Open questions include how to apply DeepLIFT to RNNs, how to compute a good reference empirically from the data, and how best to propagate importance through ‘max’ operations (as in Maxout or Maxpooling neurons) beyond simply using the gradients.

## 6. Appendix

The appendix can be downloaded at:

<http://proceedings.mlr.press/v70/shrikumar17a/shrikumar17a-sup.pdf>

## References

Bach, Sebastian, Binder, Alexander, Montavon, Grégoire, Klauschen, Frederick, Müller, Klaus-Robert, and Samek, Wojciech. On Pixel-Wise explanations for Non-Linear classifier decisions by Layer-Wise relevance propagation. *PLoS One*, 10(7):e0130140, 10 July 2015.

Chollet, François. keras. <https://github.com/fchollet/keras>, 2015.

Kheradpour, Pouya and Kellis, Manolis. Systematic discovery and characterization of regulatory motifs in encode tf binding experiments. *Nucleic acids research*, 42(5):2976–2987, 2014.

Kindermans, Pieter-Jan, Schtt, Kristof, Mller, Klaus-Robert, and Dhne, Sven. Investigating the influence of noise and distractors on the interpretation of neural networks. *CoRR*, abs/1611.07270, 2016. URL <https://arxiv.org/abs/1611.07270>.

LeCun, Yann, Cortes, Corinna, and Burges, Christopher J.C. The mnist database of handwritten digits. <http://yann.lecun.com/exdb/mnist/>, 1999.

Lundberg, Scott and Lee, Su-In. An unexpected unity among methods for interpreting model predictions. *CoRR*, abs/1611.07478, 2016. URL <http://arxiv.org/abs/1611.07478>.

Selvaraju, Ramprasaath R., Das, Abhishek, Vedantam, Ramakrishna, Cogswell, Michael, Parikh, Devi, and Batra, Dhruv. Grad-cam: Why did you say that? visual explanations from deep networks via gradient-based localization. *CoRR*, abs/1610.02391, 2016. URL <http://arxiv.org/abs/1610.02391>.

Shrikumar, Avanti, Greenside, Peyton, Shcherbina, Anna, and Kundaje, Anshul. Not just a black box: Learning important features through propagating activation differences. *arXiv preprint arXiv:1605.01713*, 2016.

Simonyan, Karen, Vedaldi, Andrea, and Zisserman, Andrew. Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034*, 2013.

Springenberg, Jost Tobias, Dosovitskiy, Alexey, Brox, Thomas, and Riedmiller, Martin A. Striving for simplicity: The all convolutional net. *CoRR*, abs/1412.6806,

2014. URL <http://arxiv.org/abs/1412.6806>.

Sundararajan, Mukund, Taly, Ankur, and Yan, Qiqi. Gradients of counterfactuals. *CoRR*, abs/1611.02639, 2016. URL <http://arxiv.org/abs/1611.02639>.

Zeiler, Matthew D. and Fergus, Rob. Visualizing and understanding convolutional networks. *CoRR*, abs/1311.2901, 2013. URL <http://arxiv.org/abs/1311.2901>.

Zhou, Jian and Troyanskaya, Olga G. Predicting effects of noncoding variants with deep learning-based sequence model. *Nat Methods*, 12:931–4, 2015 Oct 2015. ISSN 1548-7105. doi: 10.1038/nmeth.3547.

Zintgraf, Luisa M, Cohen, Taco S, Adel, Tameem, and Welling, Max. Visualizing deep neural network decisions: Prediction difference analysis. *ICLR*, 2017. URL <https://openreview.net/pdf?id=BJ5UeU9xx>.

## 7. Acknowledgements

We thank Anna Shcherbina for early experiments applying DeepLIFT to image data and beta-testing. We thank Sinhan Kang of Korea University for identifying a typing error in Section 3.6.

## 8. Funding

AS was supported by a Howard Hughes Medical Institute International Student Research Fellowship and a Bio-X Bowes Fellowship. PG was supported by a Bio-X Stanford Interdisciplinary Graduate Fellowship. AK was supported by NIH grants DP2-GM-123485 and 1R01ES025009-02.

## 9. Author Contributions

AS & PG conceptualized DeepLIFT. AS implemented DeepLIFT. AS ran experiments on MNIST. AS & PG ran experiments on genomic data. AK provided guidance and feedback. AS, PG and AK wrote the manuscript.